



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Design and Implementation of a Digital Phonocardiography System for Cardiac Sound Acquisition, Processing, and Visualization

TESI DI LAUREA MAGISTRALE IN
LABORATORY OF INTERNET OF MEDICAL THINGS

Syedaman Zargari	11109586
Ali Edareh Heidarabadi	11036028
Mobin Khatib	11114435
Amirhossein Dibaj	11109540
Mostafa Moazenkakhki	11033591
Mohammadreza Zamani	10869960
Asal Abbasnejad Fard	10974178

Advisor:
Prof. Alessandra Angelucci, Prof. Marco Cominelli

Co-advisors:

Academic year:
2025–2026

Abstract: This report presents the design and implementation of a digital phonocardiography system for cardiac sound acquisition, processing, transmission, and visualization. The project integrates a custom hardware board, embedded firmware, signal-processing algorithms, wireless communication, and a software interface for data utilization and analysis. The hardware subsystem focuses on the acquisition of heart sound signals through a cardiac sound sensor, analog signal conditioning, microcontroller integration, schematic design, and printed circuit board development. The firmware manages data acquisition, preprocessing, and communication with the software platform. The signal-processing pipeline aims to improve the quality of the acquired phonocardiographic signal and extract meaningful information related to cardiac activity. In the signal-processing evaluation, the proposed pipeline detected complete cardiac cycles from a PCG recording and extracted heart-rate, HRV-like, signal-quality, morphology, and murmur-like acoustic activity descriptors. In the signal-processing evaluation, the proposed pipeline detected complete cardiac cycles from a PCG recording and extracted heart-rate, HRV-like, signal-quality, morphology, and murmur-like acoustic activity descriptors. Finally, the software interface enables data visualization, storage, and analysis. The proposed system represents a complete prototype of an Internet of Medical Things device for cardiac sound monitoring.

Key-words: phonocardiography, heart sounds, IoMT, signal processing, PCB design, embedded systems, data visualization

1. Introduction

Cardiac auscultation is one of the most common non-invasive techniques for the preliminary assessment of heart activity. Traditional auscultation relies on a stethoscope and on the experience of the clinician. Digital phonocardiography extends this approach by acquiring heart sound signals electronically, making it possible to visualize, process, store, and analyze cardiac acoustic activity.

The aim of this project is to design and implement a Digital Phonocardiography System. The system is intended to acquire heart sounds, process the recorded signal, transmit data to an external device, and provide a software interface for visualization and analysis.

The project includes the following main parts:

- design of the hardware acquisition system and integrated board;
- implementation of the firmware for sensor data acquisition;
- signal processing for noise reduction and heart sound analysis;
- data transmission between the embedded device and the software platform;
- development of a graphical user interface for data utilization and analysis.

1.1. Motivation

Heart sounds contain useful information about cardiac mechanical activity. A digital acquisition system can support objective analysis by allowing the extraction of signal features, storage of recordings, and comparison between different acquisitions. In the context of Internet of Medical Things, a digital phonocardiography device can also become part of a connected health-monitoring platform.

1.2. Project Objectives

The main objectives of the project are:

1. to design a hardware platform suitable for heart sound acquisition;
2. to develop an integrated board including the sensor interface, microcontroller, and required passive components;
3. to implement firmware for data acquisition and communication;
4. to design a signal-processing pipeline for phonocardiographic signals;
5. to implement a software interface for visualization, storage, and analysis;
6. to test and validate the complete prototype.

The report is organized as follows.

2. System Requirements and Architecture

presents the system requirements and architecture. Section 3 describes the hardware design. Section 4 explains the firmware and acquisition strategy. Section 5 presents the signal-processing pipeline. Section 6 describes the communication protocol. Section 7 presents the software and graphical user interface. Section 8.1 discusses testing and results. Finally, Section 9 reports conclusions and possible future developments.

3. Hardware Design

This section describes the hardware design of the digital phonocardiography prototype. The objective of the hardware is to acquire heart-sound activity, transfer the acquired signal to the microcontroller, and provide a compact electronic platform for local visualization and further signal processing. The board was designed in Altium Designer, starting from the schematic design and then moving to the printed circuit board layout and three-dimensional verification.

The hardware is organized into four main functional parts: the sensing section, the processing section,

the display section, and the power-supply section. The sensing section is based on the SEN-14262 sound detector module. The processing section is based on the Arduino Nano ESP32, which is responsible for reading the sensor signal and managing the system. The display section uses an OLED module connected to the microcontroller. The power-supply section includes a USB-C MicroLipo charging module and a JST-PH two-pin battery connector.

3.1. Hardware Architecture

The architecture of the system was kept modular in order to make the prototype easier to design, test, and debug. The Arduino Nano ESP32 is placed at the center of the architecture because it is connected to the sensor, the display, and the power-related signals. This makes the microcontroller the main control unit of the board.

The SEN-14262 module is used to convert acoustic activity into an electrical signal. In this project, the sound signal is used as the input for the acquisition system. This is appropriate for a phonocardiography prototype because the useful information is contained in the shape and amplitude variations of the acquired sound waveform. The signal can then be filtered and processed in the firmware or software part of the project.

The OLED display is used to provide local visual feedback. It is connected to the Arduino Nano ESP32 through the SDA and SCL lines, which allows the display to communicate with the microcontroller using only two signal lines in addition to power and ground. This choice keeps the schematic simple and reduces the number of required PCB traces.

The power section is based on a battery-powered structure. The JST-PH two-pin connector provides the connection to the battery through the VBAT and GND nets. The USB-C MicroLipo charging module provides the charging interface. In this way, the prototype can be powered by a battery while still allowing charging through a USB-C connection.

3.2. Schematic Design

The complete schematic of the hardware system is shown in Figure 1. The circuit is built around the Arduino Nano ESP32 coming with the OLED display, the SEN-14262 sensor module, the USB-C MicroLipo charger, and the battery connector.

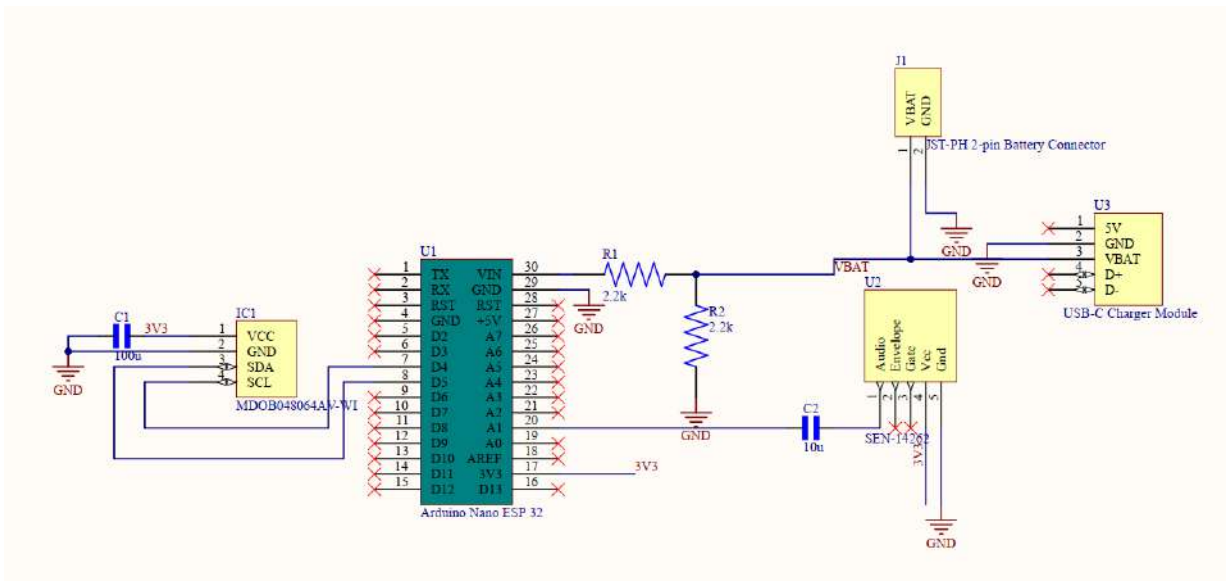


Figure 1: Complete schematic of the digital phonocardiography hardware prototype.

Capacitor C1 supports the local supply of the display and helps reduce voltage variations during operation.

The SEN-14262 module provides the sensing interface of the system. The audio output in it is routed toward the Arduino Nano ESP32 for acquisition. Capacitor C2, with a value of 10 μF , is placed in the signal path between the sensor output and the microcontroller input. This capacitor acts as a coupling capacitor and helps pass the useful varying part of the sound signal while reducing the effect of unwanted DC components. The use of the analog sound output is important because it preserves the waveform information required for later phonocardiographic signal processing.

Resistors R1 and R2 both have a value of 2.2 k Ω and are used as a voltage divider for battery-voltage measurement. Since the battery voltage can be higher than the safe input range of the microcontroller analog pin, the divider scales the VBAT voltage before it is read by the Arduino Nano ESP32. Because the two resistors have the same value, the voltage at the measurement node is approximately half of the battery voltage:

$$V_{\text{ADC}} = \frac{R_2}{R_1 + R_2} V_{\text{BAT}} = \frac{1}{2} V_{\text{BAT}}.$$

Therefore, the firmware can estimate the real battery voltage by multiplying the measured ADC voltage by two:

$$V_{\text{BAT}} \approx 2V_{\text{ADC}}.$$

This allows the system to monitor the battery level without applying the full battery voltage directly to the microcontroller input. Overall, the schematic is organized in a compact and readable way. The display is placed on the left side of the schematic, the Arduino Nano ESP32 is placed in the central area, and the sensor and power modules are placed on the right side. This organization makes the main signal flow easy to follow: the acoustic signal is acquired by the SEN-14262 module, routed to the microcontroller, and then processed or displayed according to the firmware operation.

3.3. Printed Circuit Board Layout

After completing the schematic, the circuit was transferred to the PCB layout environment in Altium Designer. The two-dimensional PCB layout is shown in Figure 2. This view shows the board outline, component placement, silkscreen labels, and the routed connections between the modules.

The power traces were made wider than the signal traces in order to reduce voltage drop, improve current-carrying capability, and increase the reliability of the power distribution path.

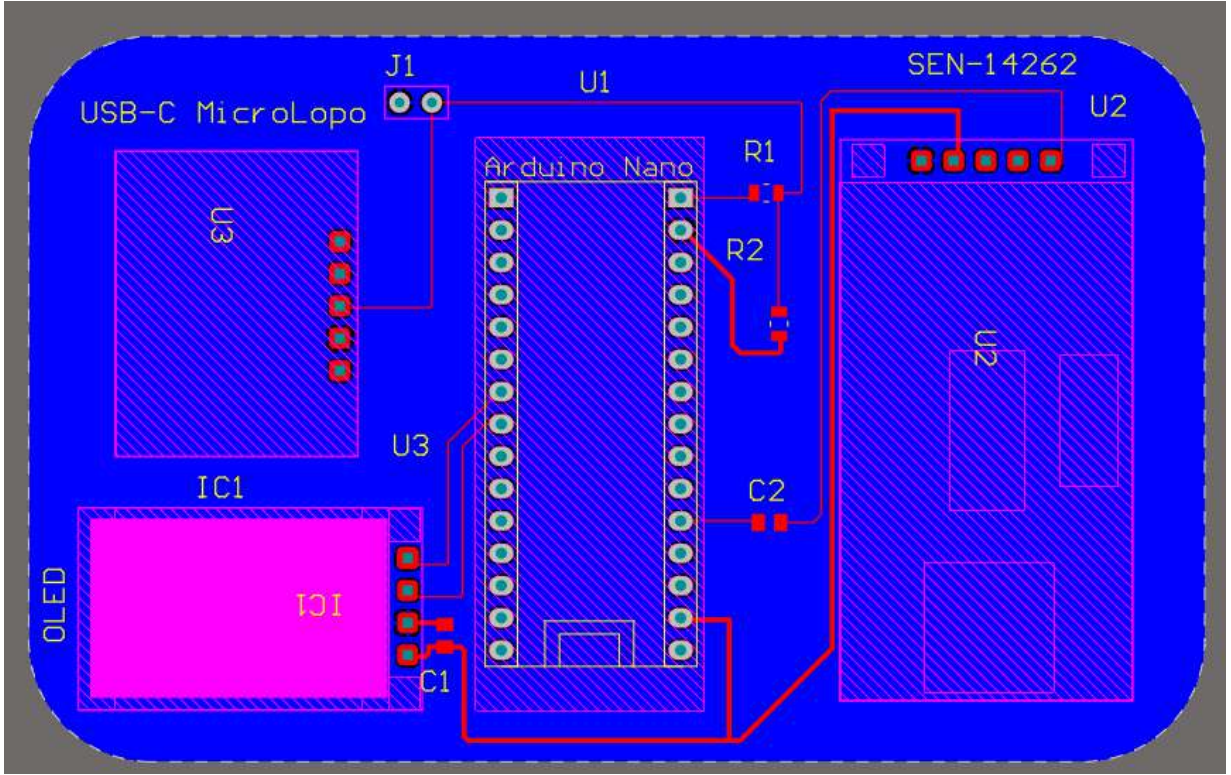


Figure 2: Two-dimensional PCB layout showing the component placement and routed connections.

The PCB has a compact rectangular shape with rounded corners. This shape improves the mechanical appearance of the prototype and avoids sharp edges. The Arduino Nano ESP32 is placed close to the center of the board. This position is practical because the microcontroller is connected to almost all the other parts of the system, including the sensor, the display, and the power-related signals.

The OLED display and the USB-C MicroLipo charging module are placed on the left side of the board. This keeps the display area and the charging connector accessible. The SEN-14262 module is placed on the right side of the PCB, separated from the display and charging section. This creates a clear functional organization: user interface and charging on the left, processing in the center, and sensing on the right.

The battery connector J1 is located near the upper part of the PCB, where it can be accessed more easily during assembly and testing. The passive components are placed close to the parts of the circuit where they are needed. C1 is positioned near the OLED supply connection, while C2 is placed along the sensor signal path. R1 and R2 are located near the power and sensor-side area of the board. The silkscreen labels identify the main components, which makes the board easier to inspect after assembly. The routing was arranged to keep the main connections short and readable. The traces from the sensor module are routed toward the Arduino Nano ESP32, while the display communication lines are routed toward the OLED module. The power and ground connections are distributed to the sensor, display, charging module, and microcontroller. Since the prototype acquires low-amplitude acoustic information, keeping the sensor path direct and avoiding unnecessary routing complexity is useful for reducing possible noise pickup.

3.4. Three-Dimensional PCB Representation

The three-dimensional PCB view was used to verify the final physical arrangement of the prototype before fabrication. The 3D view is useful because it shows not only the electrical placement of the components, but also their mechanical size and height. This is especially important in this project because the board includes several modules mounted on the same PCB.

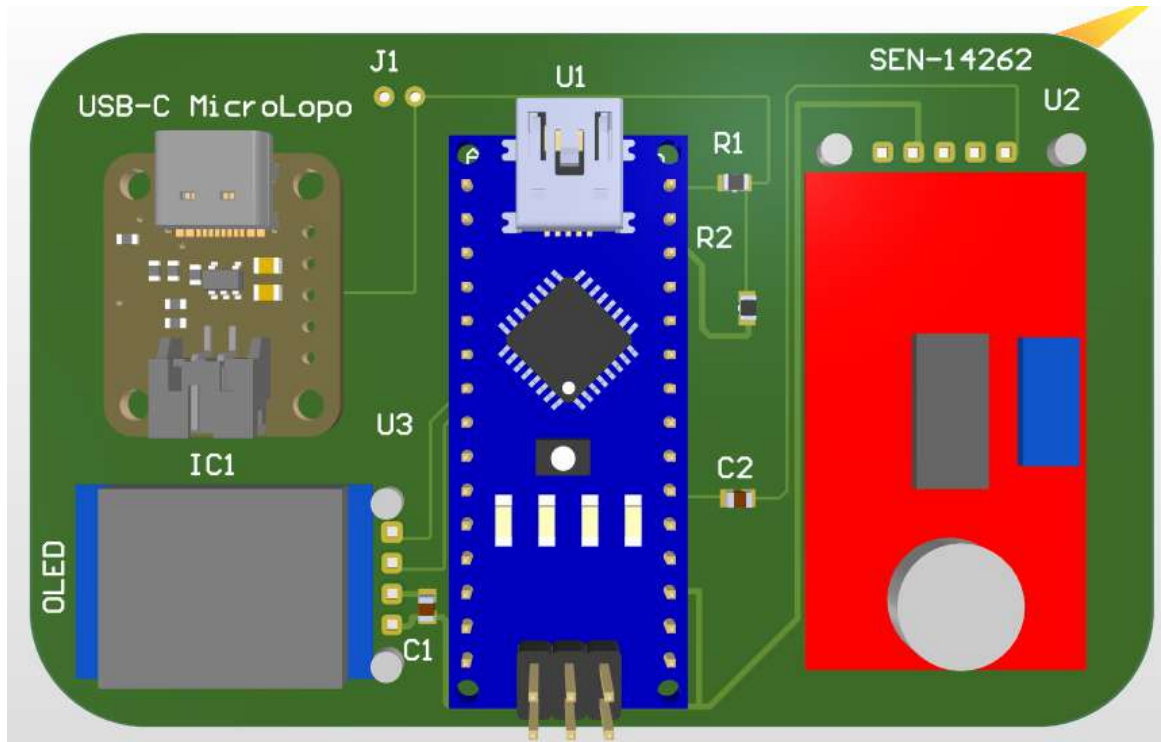


Figure 3: Top 3D view of the assembled PCB.

Figure 3 shows the top 3D view of the assembled board. The Arduino Nano ESP32 is visible in the central area. The OLED display and USB-C MicroLipo charging module are placed on the left side, while the SEN-14262 module is placed on the right side. This view confirms that the main modules are placed on the same side of the PCB and that their position is consistent with the functional organization of the schematic.

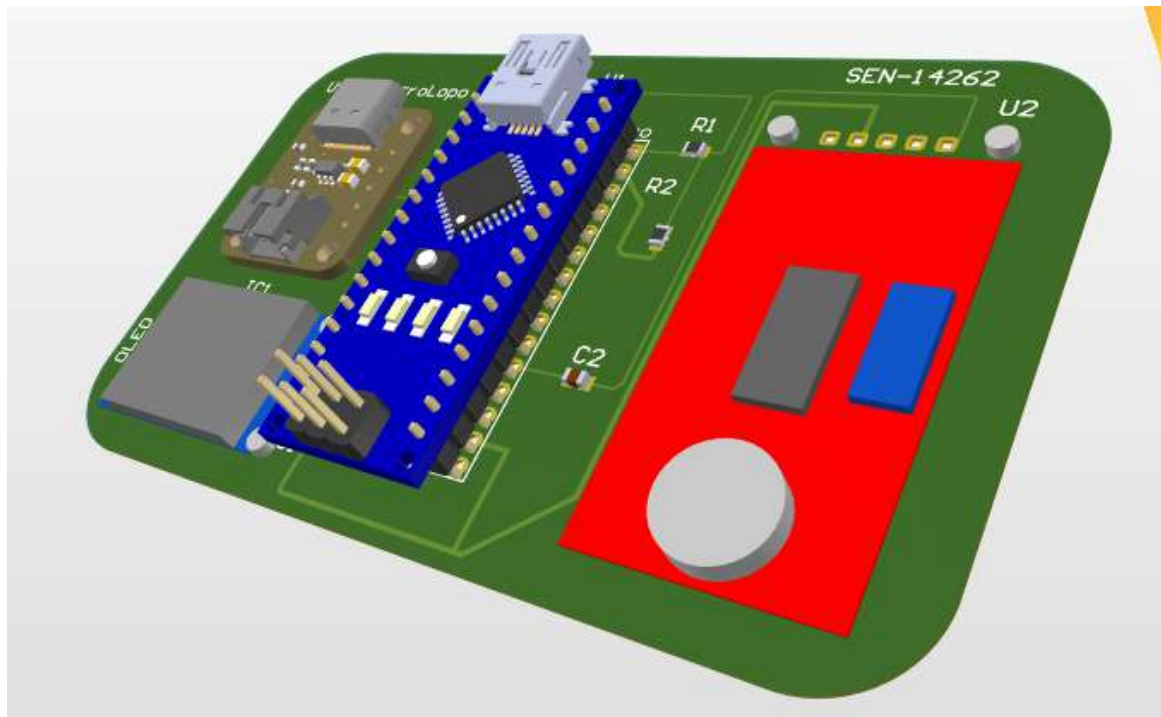


Figure 4: Perspective 3D view of the final PCB assembly.

Figure 4 shows the board from a perspective angle. This view highlights the height differences between the modules and helps check that the final assembly is mechanically realistic. The USB-C connector,

the Arduino Nano ESP32 connector area, the OLED display, and the SEN-14262 module remain visible and accessible. The 3D representation also helps evaluate whether the board can be handled easily and whether the component placement is suitable for a compact prototype.

The PCB uses a mixed assembly approach. The passive components, such as the resistors and capacitors, were implemented as surface-mount devices (SMD), while the main modules, including the Arduino Nano ESP32, OLED display, SEN-14262 sensor module, USB-C charging module, and battery connector, were mounted using through-hole or module-based footprints. This approach is suitable for a prototype because SMD resistors and capacitors reduce board area and allow compact local placement close to the required circuit nodes. At the same time, through-hole/module mounting provides better mechanical stability for larger components and makes the prototype easier to assemble, inspect, and test manually. Therefore, the combination of SMD passive components and through-hole modules represents a practical compromise between compact PCB design and prototype reliability.

3.5. Design Verification and Output Generation

After completing the PCB layout, the design was verified in Altium Designer using the Design Rule Check (DRC) tool. The final DRC result reported zero design-rule violations, indicating that the PCB layout satisfied the defined design constraints, including clearance, routing, and manufacturing-related rules.

An Output Job file was also used to generate the final project documentation and manufacturing outputs. The generated documentation includes the schematic print, PCB views, and assembly drawings. The manufacturing output package includes the Bill of Materials (BOM), Gerber files, NC Drill files, and Pick-and-Place data. The Gerber files describe the physical PCB layers required for board fabrication, while the NC Drill files define the drilled holes used for vias, through-hole pads, and connectors. The BOM lists the components used in the design, and the Pick-and-Place file defines the component positions, rotations, and board side information required for possible automated assembly.

This output generation step confirms that the hardware design was not only completed at the schematic and layout level, but was also prepared as a complete fabrication and assembly package. For bare PCB manufacturing, the Gerber and NC Drill files are the main required outputs. For assembled PCB production, the BOM and Pick-and-Place files are also required because they define which components must be used and where they must be placed on the board.

3.6. Hardware Design Choices

A first design choice was to use ready-made modules instead of designing every block from discrete components. This approach is suitable for a course prototype because it reduces design complexity and allows the system to be assembled and tested more quickly. The Arduino Nano ESP32 provides the processing platform, the SEN-14262 module provides the acoustic sensing stage, the OLED display provides local visualization, and the USB-C MicroLipo module provides a practical battery-charging interface.

A second design choice was to acquire the analog sound signal from the SEN-14262 module. This is more useful than using only a digital sound-detection output because the complete waveform is required for phonocardiographic analysis. The waveform can be filtered, segmented, and processed later by the firmware or software pipeline. This makes the hardware more suitable for signal analysis rather than only detecting the presence of sound.

A third design choice was the central placement of the Arduino Nano ESP32 on the PCB. This reduces the average distance between the microcontroller and the other modules. It also makes the board easier to route, because the display, sensor, battery connector, and charging module are arranged around the main controller.

A fourth design choice was to keep the board visually and functionally organized. The display and charging module are placed on one side of the board, while the sensing module is placed on the opposite

side. This layout makes the prototype easier to understand and inspect. The silkscreen labels also help identify the components during assembly, debugging, and presentation of the final board.

3.7. Hardware Summary

The final hardware design integrates the sensing, processing, display, and power-supply functions on a single PCB. The schematic defines the connections between the Arduino Nano ESP32, the SEN-14262 sound detector, the OLED display, the USB-C MicroLipo charging module, and the battery connector. The PCB layout places the microcontroller in the central area, the display and charging section on the left side, and the sensor module on the right side. The 3D representation confirms the physical arrangement of the assembled prototype and supports the verification of connector accessibility and module placement.

Overall, the hardware provides a compact platform for heart-sound acquisition and analysis. It is suitable for an educational digital phonocardiography prototype and can be integrated with the firmware, signal-processing pipeline, data-transmission protocol, and graphical user interface described in the other sections of the report.



(a) Final box enclosure



(b) Digital phonocardiograph used for data processing

Figure 5: Final product implementation.

4. Firmware and Data Acquisition

4.1. Firmware Overview

The firmware implements the Arduino Nano ESP32 as the embedded acquisition and communication unit of the digital phonocardiography prototype. Its main role is to acquire the analog output of the SEN-14262 sound sensor, buffer the raw samples, and stream them to an external software platform through Bluetooth Low Energy (BLE). The firmware also monitors the battery voltage and shows the device status on a small OLED display.

The implemented behavior is based on a BLE-controlled acquisition session. After power-up, the device advertises itself as `PCG_Monitor_Raw`. When a BLE central device connects, it can send a start command containing the sampling parameters, oversampling factor, batch size, acquisition duration, and patient name. The firmware then starts a timer-driven acquisition task, samples the PCG signal, groups samples into small batches, and transmits each batch using BLE notifications.

The firmware does not perform high-level PCG analysis such as S1/S2 detection, filtering, segmentation, or feature extraction. Instead, it streams raw ADC samples to the external software, where the main signal-processing pipeline can be applied.

4.2. Main Hardware and Interfaces

The main hardware elements inferred from the firmware are summarized in Table 1.

Component	Code reference	Function
Arduino Nano ESP32	Main MCU	Executes the acquisition logic, BLE communication, display update, and battery monitoring.
SEN-14262 sound sensor	SIGNAL_PIN = A1	Provides the analog cardiac sound signal to the ADC input.
Battery voltage divider	BATTERY_PIN = A0	Allows the firmware to estimate the battery level from a scaled analog voltage.
OLED display	U8G2_SSD1306_48X64	Shows local status information such as battery percentage, BLE advertising, connection, and analysis state.
BLE central device (Arduino ESP32 NANO)	BLE GATT client	Sends the start command and receives streamed PCG samples.

Table 1: Main hardware and firmware interfaces.

The display uses the hardware I2C bus. The code comment indicates SDA = A4 and SCL = A5. The sound signal and battery voltage are acquired through analog input pins. No SPI, Wi-Fi, or external digital sensor bus is used for the PCG acquisition.

4.3. Code Structure

The firmware is organized around the following main blocks:

- **Libraries:** BLE communication is implemented using BLEDevice.h, BLEServer.h, BLEUtils.h, and BLE2902.h. The OLED is controlled using U8g2lib.h over I2C through Wire.h.
- **State machine:** the device uses four states: STATE_INITIALIZING, STATE_IDLE, STATE_CONNECTED, and STATE_ANALYZING.
- **BLE setup:** the firmware creates one BLE service and one characteristic. The characteristic supports read, write without response, and notify operations.
- **Command parser:** parseStartCommand() decodes the BLE start command and updates acquisition parameters.
- **Sampling task:** samplingTask() is a FreeRTOS task that waits for timer notifications, reads the ADC, performs oversampling, and fills a sample batch.
- **Timer interrupt:** onTimer() does not read the ADC directly. It only notifies the sampling task, which is safer on ESP32 because analogRead() should not be called inside a hardware timer ISR.
- **Main loop:** loop() starts the timer when needed, checks the acquisition timeout, drains the sample queue, sends BLE notifications, and updates the OLED display every 150 ms.
- **Display helpers:** functions such as updateDisplay(), drawBT(), drawSpinner(), and drawTopPanel() manage the user interface.
- **Battery helper:** batteryPercent() averages 16 battery-voltage readings and converts the result to an approximate percentage.

4.4. Communication Methods

The firmware uses the following communication methods:

- **BLE:** main wireless communication method. The device works as a BLE GATT server. A BLE central writes commands to the characteristic and receives sensor data through notifications.
- **I2C:** used only for the OLED display.
- **Serial:** used for debugging messages at 115200 baud.

- **Analog ADC:** used to acquire the PCG sensor output and the battery-voltage monitor.

No Wi-Fi, Bluetooth Classic, SPI, or UART-based sensor communication is implemented in the uploaded firmware.

4.5. Data Acquisition and Packetization

The sensor data path starts from the analog output of the SEN-14262 sound sensor. This analog waveform is connected to A1, where it is digitized by the ESP32 ADC. The ADC resolution is set to 12 bits using:

```
analogReadResolution(12);
```

Each output sample is produced by averaging several fast ADC readings:

$$x[k] = \frac{1}{N_{os}} \sum_{i=1}^{N_{os}} x_i[k], \quad (1)$$

where N_{os} is the oversampling count. The default oversampling count is 8, and the command parser constrains it between 1 and 32. The averaged samples are stored in a SampleBatch structure:

```
struct SampleBatch {
uint16_t samples[MAX_BATCH_SIZE];
uint8_t count;
};
```

The maximum batch size is 6 samples. Therefore, each BLE data notification contains at most 12 bytes of sample payload.

4.5.1 BLE Start Command

The BLE start command is sent by the external software to the same BLE characteristic used for streaming. The firmware expects a minimum command length of 29 bytes. The packet format is shown in Table 2.

Bytes	Size	Field	Description
0	1 byte	Command type	Must be 0x01 for START.
1–4	4 bytes	Sample rate	Unsigned 32-bit little-endian value, constrained to 100–5000 Hz.
5–6	2 bytes	Oversampling count	Unsigned 16-bit little-endian value, constrained to 1–32.
7–8	2 bytes	Batch size	Unsigned 16-bit little-endian value, constrained to 1–6.
9–12	4 bytes	Analysis time	Unsigned 32-bit little-endian duration in seconds.
13–28	16 bytes	Patient name	Character field. The firmware stores up to 15 characters and appends a null terminator.

Table 2: BLE START command format.

4.5.2 BLE Sample Notifications

The PCG data stream is sent using BLE notifications. Each notification contains only raw ADC samples, without an application-level header. The transmitted value is created using:

```
pCharacteristic->setValue((uint8_t*)outBatch.samples,
outBatch.count * sizeof(uint16_t));
pCharacteristic->notify();
```

Thus, the BLE notification payload is a byte array containing consecutive `uint16_t` samples. Since the ESP32 is little-endian, each sample is transmitted in little-endian byte order.

Packet element	Implementation in current firmware
Header	Not present. BLE notification boundary acts as the only frame boundary.
Payload	Raw ADC samples, each stored as <code>uint16_t</code> .
Payload length	$2 \times \text{batchSize}$ bytes. Maximum 12 bytes.
Timestamp	Not present. The receiver must reconstruct time from the configured sample rate.
Sequence number	Not present. Lost BLE notifications cannot be detected by the application.
Checksum or CRC	Not present. Reliability depends on the BLE stack.
Delimiter or end marker	Not present. End of acquisition is not explicitly transmitted.

Table 3: Current BLE sample notification format.

The resulting data flow is:

1. The BLE central connects to the ESP32.
2. The BLE central writes a **START** command.
3. The ESP32 configures sample rate, oversampling, batch size, acquisition duration, and patient name.
4. The hardware timer periodically wakes the sampling task.
5. The sampling task reads the analog PCG signal from A1.
6. Several ADC readings are averaged to produce one sample.
7. Samples are grouped into a `SampleBatch`.
8. The completed batch is pushed into a FreeRTOS queue.
9. The main loop drains the queue and sends each batch as a BLE notification.
10. The external software receives the byte array and reconstructs the raw PCG waveform.

4.6. Main Control Algorithm

The firmware behavior is controlled by a simple state machine. In the idle state, the device advertises itself over BLE. After a BLE connection, it waits for a **START** command. During analysis, a hardware timer defines the sampling instants and the sampling task performs ADC acquisition. When the selected analysis time expires, the firmware stops the timer and returns to the connected state.

State	Meaning
STATE_INITIALIZING	Peripherals and BLE are being initialized.
STATE_IDLE	Device is advertising and waiting for a BLE connection.
STATE_CONNECTED	BLE central is connected, but acquisition has not started.
STATE_ANALYZING	Timer-driven sampling and BLE streaming are active.

Table 4: Firmware state machine.

4.7. Diagrams

The following diagrams use a layered architecture style similar to the reference notes. Light gray areas indicate system regions, colored blocks indicate functional modules, black arrows represent data paths, and red arrows represent control or command paths.

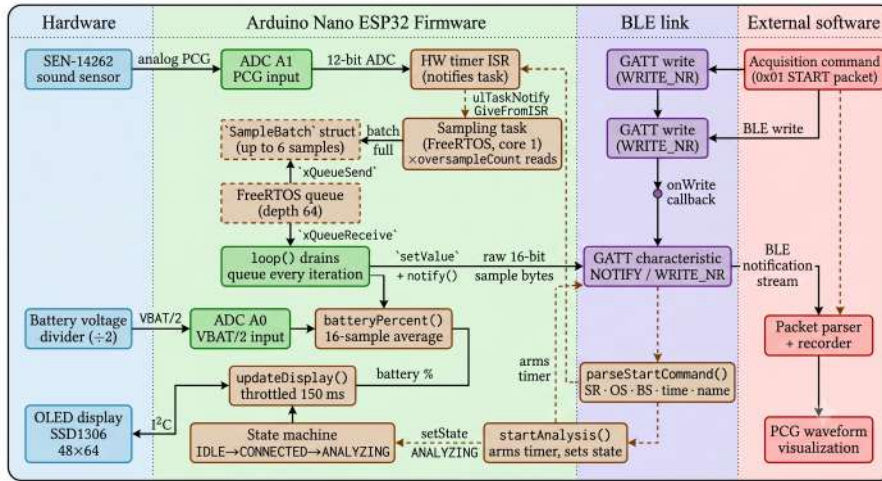


Figure 6: Layered firmware architecture and data path

4.7.1 Firmware Architecture and Data Path

4.7.2 Firmware Control Sequence

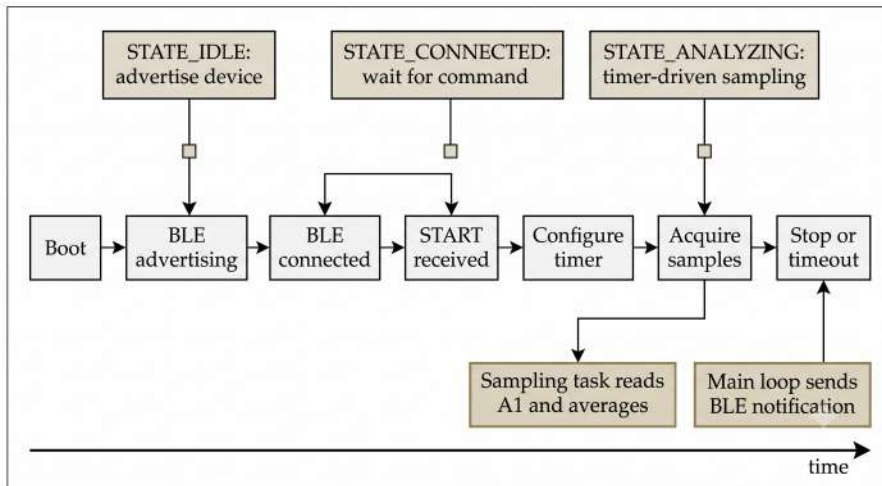
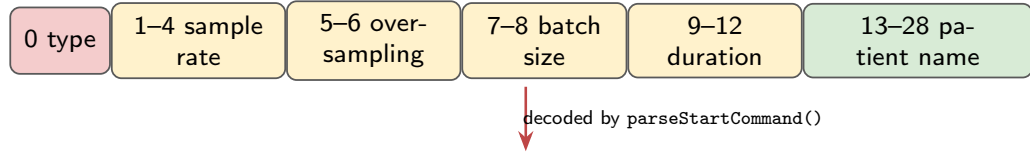


Figure 7: Firmware control sequence from boot to BLE connection, acquisition, notification, and acquisition stop.

4.7.3 BLE Command and Data Packet Format

START command written by the software



BLE sample notification sent by the firmware



Current payload: raw little-endian ADC samples only. No application header, timestamp, sequence number, checksum, or explicit end marker.

Figure 8: BLE command and sample-notification formats used by the current firmware.

4.7.4 Inferred Wiring and Pin Connection Diagram

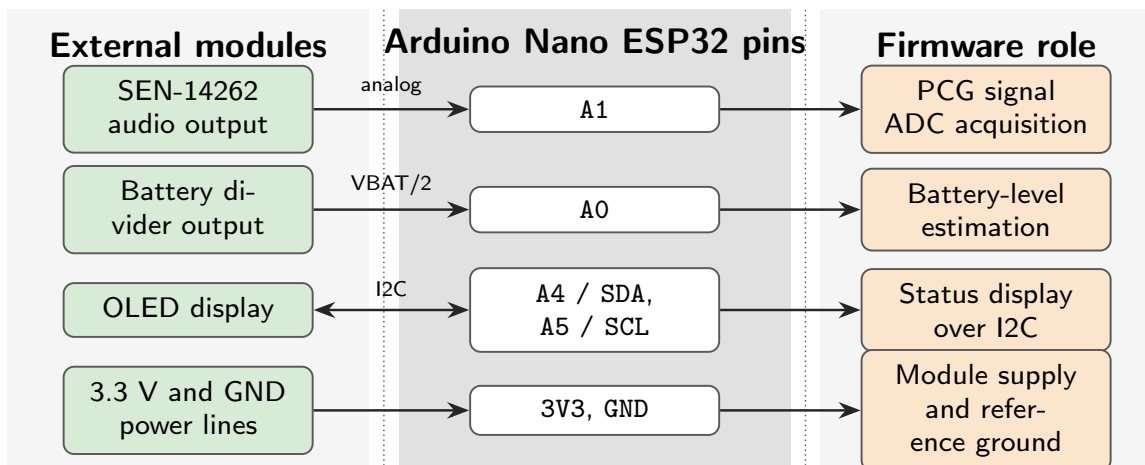


Figure 9: Inferred wiring and firmware pin usage based on the code definitions.

5. Signal Processing

5.1. Phonocardiographic Signal Characteristics

The main objective of the signal-processing stage is to transform the raw phonocardiographic (PCG) recording into a structured signal that can be analysed automatically. The raw PCG signal contains the acoustic activity of the heart, but it may also include environmental noise, sensor movement, poor contact noise, breathing sounds, and electrical interference. Therefore, preprocessing and filtering are required before detecting the main heart sound components.

In this project, the PCG processing pipeline consists of the following main steps: loading the raw WAV recording, converting it to a mono signal when needed, removing the DC component, normalizing the amplitude, applying notch and band-pass filtering, reducing noise using wavelet denoising and smoothing, extracting the Shannon energy envelope, detecting S1 and S2 peaks, segmenting the signal into cardiac phases, and extracting cycle-based features. The cardiac cycle is represented using four main states:

$$S1 \rightarrow \text{Systole} \rightarrow S2 \rightarrow \text{Diastole} \quad (2)$$

Here, S1 and S2 are used as reference events for cardiac cycle detection. S1 is mainly associated with the closure of the mitral and tricuspid valves, while S2 is mainly associated with the closure of the aortic and pulmonary valves. Once S1 and S2 are detected, the systolic and diastolic intervals can be estimated and analyzed separately. Before applying the filtering pipeline, the original raw PCG waveform was inspected in the time domain. This step is useful for observing the general amplitude range.

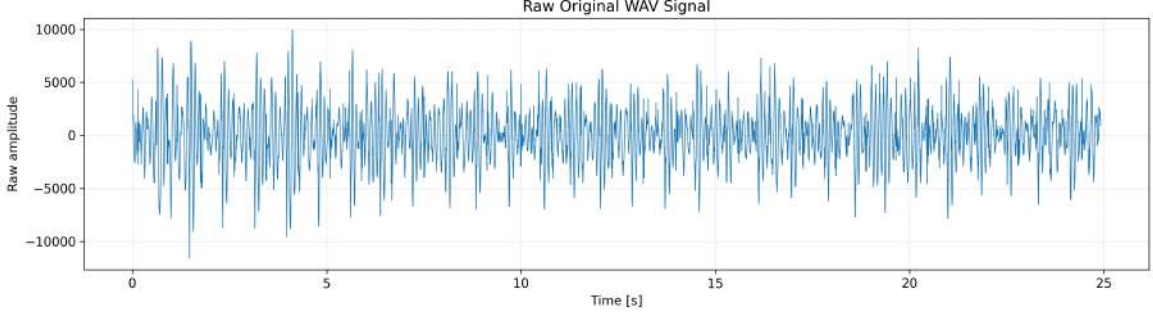


Figure 10: Original raw PCG waveform before preprocessing and filtering.

5.2. Noise Sources and Preprocessing

The raw PCG recording is not perfectly clean because it is acquired in a practical environment. Several types of noise may be present in the signal, such as background acoustic noise, sensor movement, poor sensor contact, electronic noise, breathing sounds, and power-line interference. These noise sources can make S1 and S2 detection difficult if the raw signal is used directly.

Therefore, the first step is to preprocess the audio recording. The WAV file is loaded and converted into a mono signal if it contains more than one channel. The signal is then converted to floating-point values. After that, the DC component is removed by subtracting the mean value of the signal:

$$x[n] = x_{\text{raw}}[n] - \frac{1}{N} \sum_{i=0}^{N-1} x_{\text{raw}}[i] \quad (3)$$

Where $x_{\text{raw}}[n]$ is the original signal and N is the total number of samples. The signal is then normalized as

$$x_{\text{norm}}[n] = \frac{x[n]}{\max(|x[n]|) + \epsilon} \quad (4)$$

This normalization makes the amplitude scale more stable and prepares the PCG signal for filtering and envelope extraction.

5.3. Filtering and Denoising

After preprocessing, the signal is filtered to reduce unwanted frequency components while preserving the main heart sound activity. In this project, the useful PCG frequency content is mainly considered between 25 Hz and 200 Hz. Therefore, a band-pass filter is applied in this range. The filtering pipeline consists of four main steps:

- notch filtering at 50 Hz, 100 Hz, and 150 Hz to reduce power-line interference and its harmonics;
- band-pass filtering between 25 Hz and 200 Hz;
- wavelet denoising to reduce noise while preserving transient heart sound peaks;
- Savitzky–Golay smoothing to obtain a smoother final signal.

The notch filters are applied at:

$$f_{\text{notch}} = 50, 100, 150 \text{ Hz} \quad (5)$$

A general digital filter can be represented as:

$$y[n] = \sum_{k=0}^M b_k x[n-k] - \sum_{k=1}^N a_k y[n-k] \quad (6)$$

Where $x[n]$ is the input signal, $y[n]$ is the filtered output, and a_k and b_k are the filter coefficients. Figure 11 shows the effect of the preprocessing and filtering pipeline. Compared with the normalized input signal, the filtered PCG signal is smoother and the main heart sound events are more clearly visible.

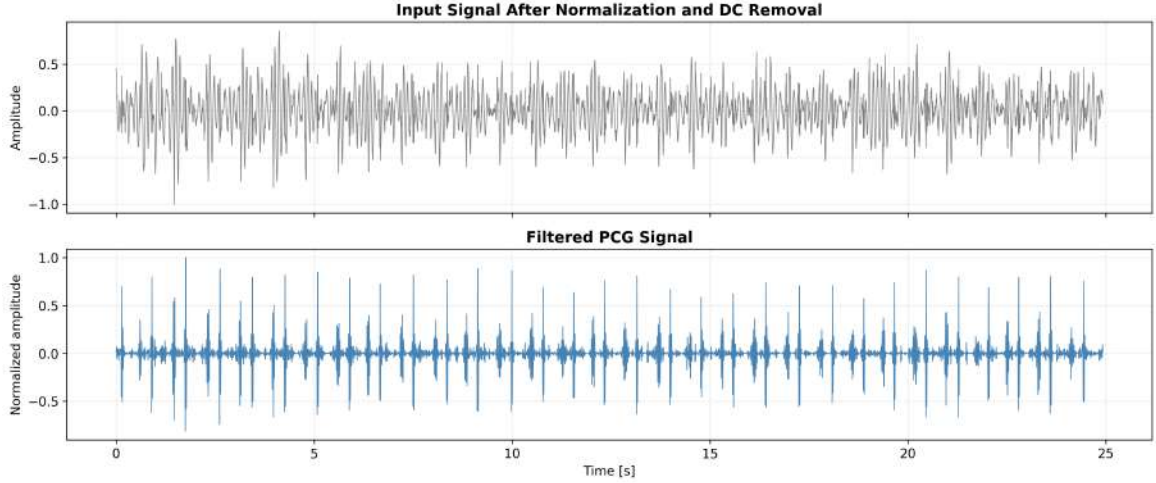


Figure 11: Comparison between the normalized PCG signal and the filtered PCG signal after notch filtering, band-pass filtering, wavelet denoising, and smoothing.

5.4. Envelope Extraction and S1/S2 Peak Detection

After filtering, the PCG waveform still contains fast oscillations, which makes direct peak detection difficult. For this reason, an envelope is extracted from the filtered signal. In this project, the Shannon energy envelope is used because it emphasizes the main heart sound events and reduces the influence of small-amplitude fluctuations.

The Shannon energy is calculated as:

$$E_s[n] = -x^2[n] \log(x^2[n] + \epsilon) \quad (7)$$

where $x[n]$ is the normalized filtered PCG signal and ϵ is a small constant used to avoid numerical problems.

The Shannon energy signal is then smoothed using a low-pass filter:

$$e[n] = \text{LPF}\{E_s[n]\} \quad (8)$$

In the implementation, the envelope cutoff frequency is set to 8 Hz. This produces a smooth envelope that follows the main heart sound activity and makes S1 and S2 peak detection easier.

Candidate peaks are detected from the Shannon envelope using an adaptive threshold. The threshold is based on the 75th percentile of the envelope:

$$T = P_{75}(e[n]) \quad (9)$$

Only peaks higher than this threshold are considered as candidate heart sound events. A minimum peak distance is also applied to avoid detecting the same heart sound more than once.

After peak detection, physiological timing rules are used to classify the detected peaks as S1 or S2. The accepted range for the S1–S2 interval is:

$$0.07 \leq t_{\text{S1-S2}} \leq 0.50 \text{ s} \quad (10)$$

and the accepted range for the S2–next S1 interval is:

$$0.18 \leq t_{S2-S1} \leq 1.40 \text{ s} \quad (11)$$

Peaks that satisfy these timing constraints are accepted as valid S1–S2 pairs. Peaks that do not satisfy the constraints are kept as unassigned peaks. This step provides the reference points required for cardiac cycle segmentation.

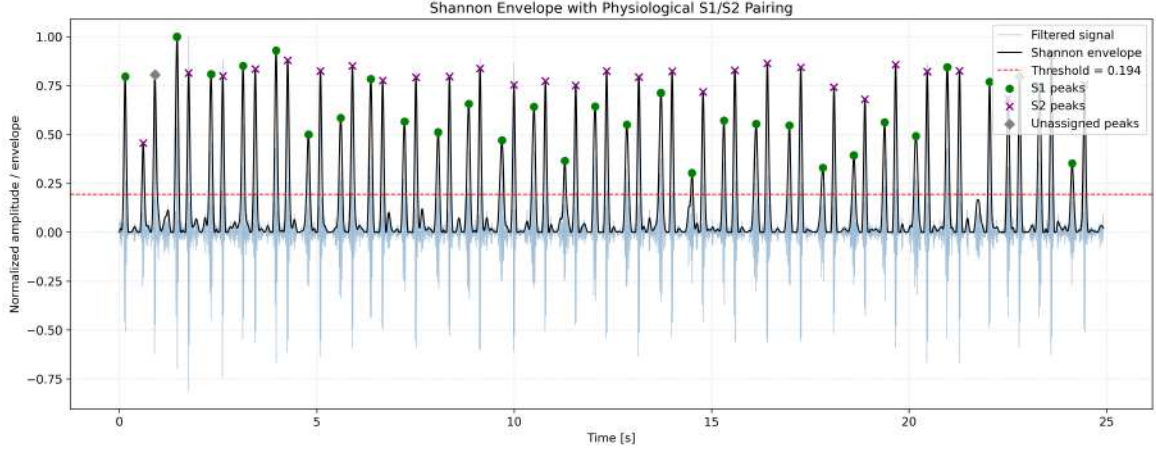


Figure 12: Shannon energy envelope with detected S1 and S2 peaks using physiological timing constraints.

As shown in Figure 12, the Shannon envelope highlights the main heart sound events and allows the S1 and S2 peaks to be detected using the selected threshold and timing constraints. The unassigned peaks correspond to detected peaks that did not satisfy the physiological pairing rules.

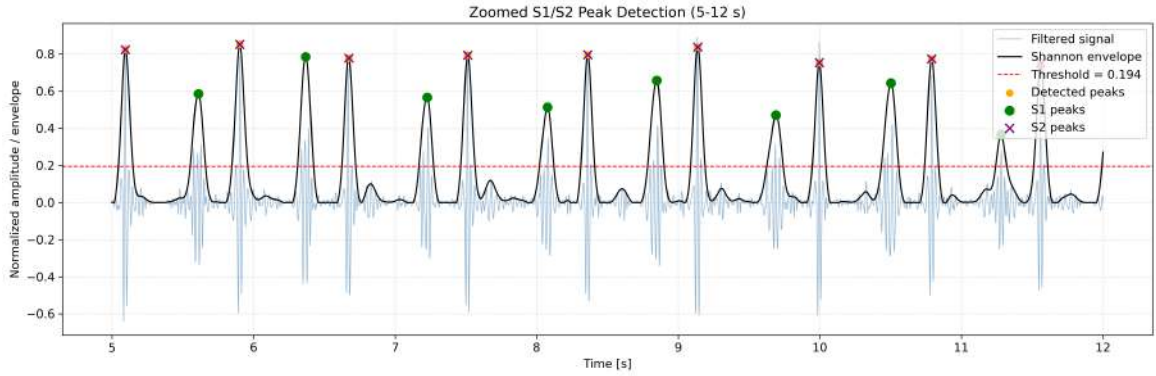


Figure 13: Zoomed view of Shannon envelope-based peak detection, showing the classification of detected peaks into S1 and S2 within a selected time interval.

The zoomed representation is particularly useful for visually validating the S1/S2 assignment logic. Since the interval between S1 and S2 is typically shorter than the interval between S2 and the following S1, the local timing pattern shown in Figure 13 helps confirm that the detected peaks are consistent with physiological expectations.

5.5. Cardiac Cycle Segmentation

After detecting the S1 and S2 peaks, the PCG signal is segmented into four main cardiac states: S1, systole, S2, and diastole. This segmentation is necessary because the later features are extracted separately from different parts of the cardiac cycle.

The boundaries of S1 and S2 are estimated around each detected peak using the Shannon envelope. For each peak, the algorithm searches to the left and right until the envelope falls below a relative percentage of the peak height. In this implementation, the relative boundary level is set to 40% of the peak value:

$$e[n] < 0.4e[p] \quad (12)$$

where $e[n]$ is the envelope value and p is the detected peak location.

Once the S1 and S2 boundaries are estimated, the interval between S1 and S2 is labelled as systole, while the interval between S2 and the next S1 is labelled as diastole. Therefore, each complete cardiac cycle follows the sequence:

$$S1 \rightarrow \text{Systole} \rightarrow S2 \rightarrow \text{Diastole} \quad (13)$$

This four-state segmentation allows the temporal and energy-based properties of each cardiac phase to be analysed separately.

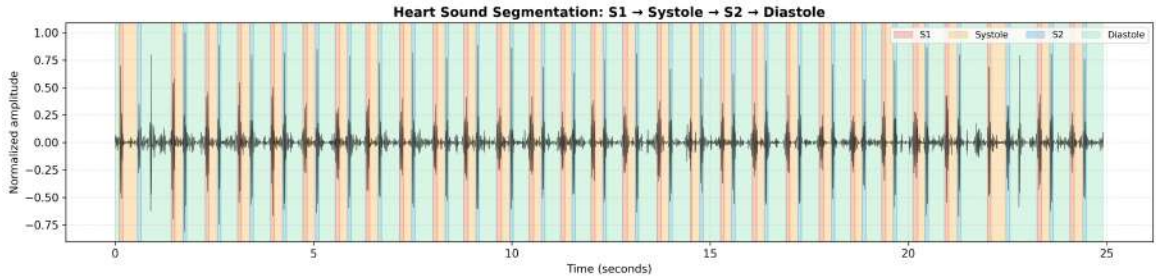


Figure 14: Four-state segmentation of the PCG signal into S1, systole, S2, and diastole based on the detected S1 and S2 peaks.

Figure 14 shows the segmented PCG signal. The segmentation provides the basis for cycle-level feature extraction since each accepted cardiac cycle is analysed using its S1, systolic, S2, and diastolic regions.

5.6. Cycle-Level Feature Extraction

After segmenting the PCG signal into S1, systole, S2, and diastole, cycle-level features are extracted from each complete cardiac cycle. A cycle is accepted only if it follows the expected sequence:

$$S1 \rightarrow \text{Systole} \rightarrow S2 \rightarrow \text{Diastole} \quad (14)$$

For each accepted cycle, temporal, energy-based, and spectral features are calculated. Temporal features describe the duration of the main cardiac phases, while energy-based features describe the relative strength of S1, S2, systole, and diastole. Spectral features provide additional information about the frequency content of the heart sounds. The heart rate for each cycle is estimated from the cycle duration:

$$HR = \frac{60000}{T_{\text{cycle}}} \quad (15)$$

Where T_{cycle} is the full cardiac cycle duration in milliseconds. The RMS value is used to estimate the energy level of each signal segment:

$$RMS(x) = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x^2[n]} \quad (16)$$

The S1/S2 amplitude ratio is computed as:

$$R_{S1/S2} = \frac{RMS_{S1}}{RMS_{S2} + \epsilon} \quad (17)$$

This ratio is useful for describing the relative strength of the two main heart sounds and for identifying cycles where one of the heart sounds is unusually weak or strong. The main extracted features are summarized in Table 5.

Table 5: Main cycle-level features extracted from each accepted cardiac cycle.

Feature group	Extracted features
Temporal features	S1 duration, S2 duration, systolic duration, diastolic duration, full cycle duration, and heart rate
Energy-based features	S1 RMS, S2 RMS, S1/S2 amplitude ratio, energy concentration, systolic noise ratio, and diastolic noise ratio
Spectral features	Spectral centroid, zero-crossing rate, and kurtosis
Cepstral features	MFCC coefficients extracted from the accepted cardiac cycle
Ratio-based features	Systole-to-diastole duration ratio and S1/S2 amplitude ratio

5.7. Rule-Based Cycle Screening and Robust Outlier Analysis

After extracting cycle-level features, a rule-based screening step is applied to identify cycles that deviate from expected physiological or signal-quality ranges. This step is not used as a medical diagnosis. Instead, it is used as a signal-processing quality check to flag cycles that may require further inspection.

Each accepted cardiac cycle is checked using predefined ranges for features such as S1 duration, S2 duration, systolic duration, diastolic duration, S1/S2 amplitude ratio, and heart rate. If one or more features fall outside the expected range, the cycle is marked as a rule-flagged cycle. For example, the heart rate range used in the screening step is:

$$45 \leq HR \leq 180 \text{ bpm} \quad (18)$$

Similarly, the S1 and S2 durations are checked using expected duration ranges:

$$40 \leq T_{S1} \leq 200 \text{ ms} \quad (19)$$

$$40 \leq T_{S2} \leq 150 \text{ ms} \quad (20)$$

In addition to rule-based screening, a robust outlier analysis is performed using the median absolute deviation (MAD). This allows the pipeline to detect cycles that are very different from the rest of the recording, even if they are still within broad physiological limits.

The robust z-score is calculated as:

$$z_{\text{robust}} = 0.6745 \frac{x - \text{median}(x)}{\text{MAD} + \epsilon} \quad (21)$$

where x is the feature value, $\text{median}(x)$ is the median of that feature across cycles, and MAD is the median absolute deviation.

A feature value is considered a strong outlier if:

$$|z_{\text{robust}}| > 3.5 \quad (22)$$

The combination of rule-based screening and robust outlier analysis helps identify uncertain or unusual cardiac cycles before interpreting the final signal-derived results.

5.8. Heart Rate and Short-Window HRV-Like Analysis

After S1 peak detection, the heart rate can be estimated from the time intervals between consecutive S1 peaks. In this project, the S1–S1 intervals are used as PCG-derived cycle intervals:

$$NN_i = t_{S1,i+1} - t_{S1,i} \quad (23)$$

where $t_{S1,i}$ is the time location of the i -th detected S1 peak. The instantaneous heart rate for each interval is computed as:

$$HR_i = \frac{60000}{NN_i} \quad (24)$$

where NN_i is measured in milliseconds.

Several short-window HRV-like metrics are then calculated from the S1–S1 intervals, including the mean NN interval, median NN interval, SDNN, RMSSD, NN50, pNN50, and the coefficient of variation of NN intervals.

The SDNN is calculated as the standard deviation of the valid S1–S1 intervals:

$$SDNN = std(NN_i) \quad (25)$$

The RMSSD is calculated from the differences between consecutive intervals:

$$RMSSD = \sqrt{\frac{1}{M-1} \sum_{i=1}^{M-1} (NN_{i+1} - NN_i)^2} \quad (26)$$

Since these intervals are derived from PCG S1 detections rather than ECG R-peaks, the resulting metrics are reported as short-window HRV-like metrics. They should not be interpreted as clinical ECG-based HRV estimates.

To reduce the effect of missed or uncertain S1 detections, a robust artifact-rejection step is also applied. Intervals that are too far from the median-based accepted range are rejected before recalculating the robust HRV-like metrics.

5.9. Spectrogram and Signal Quality Analysis

In addition to the time-domain analysis, the filtered PCG signal is also analysed in the frequency domain. A spectrogram is used to show how the frequency content of the PCG signal changes over time. This representation is useful because heart sound components are short acoustic events and their frequency content may vary across different cardiac phases.

The spectrogram is computed from the filtered PCG signal and displayed together with the cardiac segmentation overlay. This allows the relationship between the time-frequency energy distribution and the detected cardiac phases to be visually inspected.

The pipeline also computes several signal quality descriptors, including the out-of-band energy ratio before and after filtering, the out-of-band energy reduction, dominant frequency, spectral centroid, spectral bandwidth, spectral entropy, and band-energy distribution. These features are used only as signal descriptors and quality indicators, not as diagnostic markers. The band-energy distribution is calculated over three frequency ranges:

- low band: 25–60 Hz;
- mid band: 60–120 Hz;
- high band: 120–200 Hz.

This analysis helps verify whether most of the filtered PCG energy remains concentrated in the expected frequency range after preprocessing and filtering.

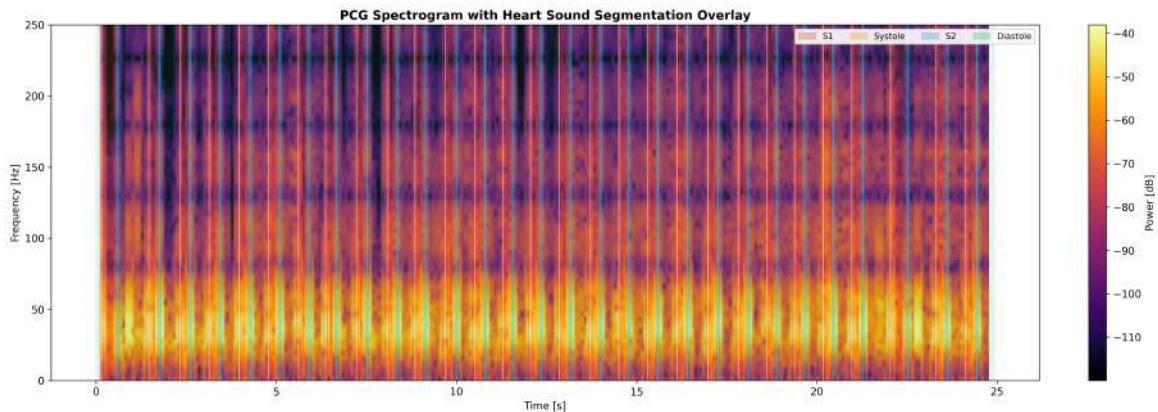


Figure 15: Spectrogram of the filtered PCG signal with heart sound segmentation overlay. The figure shows the time-frequency energy distribution of the recording together with the detected S1, systolic, S2, and diastolic regions.

As shown in Figure 15, the spectrogram provides a time-frequency view of the filtered PCG signal. The segmentation overlay helps relate the detected cardiac phases to the corresponding acoustic energy patterns.

5.10. Cycle Morphology Consistency

To evaluate the consistency of the detected cardiac cycles, an additional morphology analysis is performed. In this step, each accepted cardiac cycle is aligned around its S1 location. A fixed time window is extracted around S1, and each cycle snippet is normalized using z-score normalization. This allows the waveform shapes to be compared independently of their absolute amplitude. After alignment, a mean cycle template is computed from all accepted cycles:

$$\bar{x}_{\text{template}}[n] = \frac{1}{K} \sum_{i=1}^K x_i[n] \quad (27)$$

where K is the number of aligned cycles and $x_i[n]$ is the normalized waveform of the i -th cycle. The similarity between each cycle and the mean template is then measured using correlation:

$$\rho_i = \text{corr}(x_i[n], \bar{x}_{\text{template}}[n]) \quad (28)$$

A low correlation value indicates that the morphology of a cycle differs from the average cycle shape. This does not indicate a clinical abnormality. It only shows that the waveform shape of that cycle is less consistent with the mean template, which may be caused by noise, sensor movement, segmentation uncertainty, or natural variability in the recording.

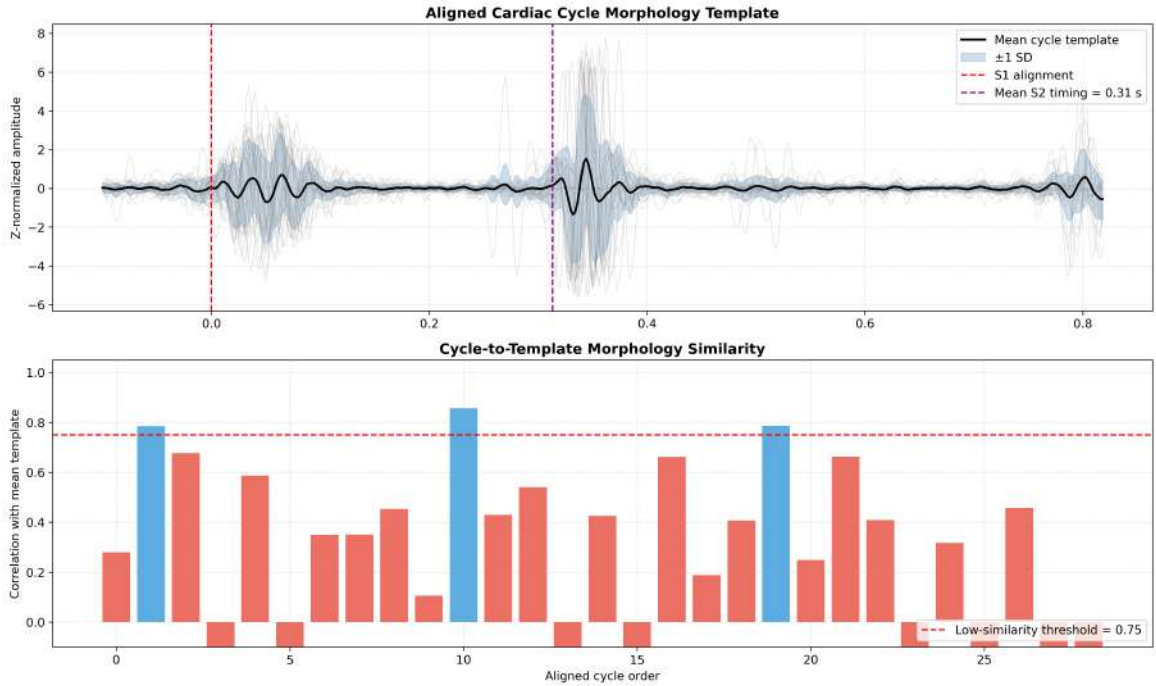


Figure 16: Cycle morphology consistency analysis. The upper plot shows aligned cardiac cycles around S1 and the mean cycle template, while the lower plot shows the correlation of each cycle with the mean template.

Figure 16 provides a visual assessment of cycle-to-cycle waveform consistency. Cycles with low template correlation are treated as low-similarity cycles and are used only as signal-quality indicators, not as diagnostic outputs.

5.11. Murmur-Like Acoustic Activity Analysis

The final part of the signal-processing pipeline checks for murmur-like acoustic activity candidates. In PCG recordings, a murmur-like pattern may appear as sustained acoustic energy during systole or diastole. However, in this project, this analysis is only used as a signal-derived acoustic pattern detector and not as a clinical diagnostic tool.

For each accepted cardiac cycle, the systolic and diastolic signal segments are analysed separately. A reference heart-sound energy level is first calculated using the RMS values of S1 and S2:

$$RMS_{\text{ref}} = \frac{RMS_{S1} + RMS_{S2}}{2} \quad (29)$$

The systolic acoustic activity ratio is then calculated as:

$$R_{\text{sys}} = \frac{RMS_{\text{systole}}}{RMS_{\text{ref}} + \epsilon} \quad (30)$$

Similarly, the diastolic acoustic activity ratio is calculated as:

$$R_{\text{dia}} = \frac{RMS_{\text{diastole}}}{RMS_{\text{ref}} + \epsilon} \quad (31)$$

A cycle is marked as having increased acoustic activity if the corresponding ratio is higher than the selected threshold:

$$R \geq 0.35 \quad (32)$$

The systolic interval is further divided into early, middle, and late parts. This allows the method to describe the timing of systolic activity as early systolic-like, mid systolic-like, late systolic-like, or holosystolic-like. The same idea is applied to the diastolic interval, with special attention to mid-diastolic and late-diastolic acoustic activity.

It is important to note that the detected patterns are not interpreted as medical diagnoses. They only indicate that the signal contains increased acoustic energy in specific cardiac phases. Such activity may also be caused by noise, sensor movement, imperfect segmentation, or recording artifacts.

5.12. Signal Processing Pipeline Summary

The complete PCG signal-processing workflow is summarized in Figure 17.

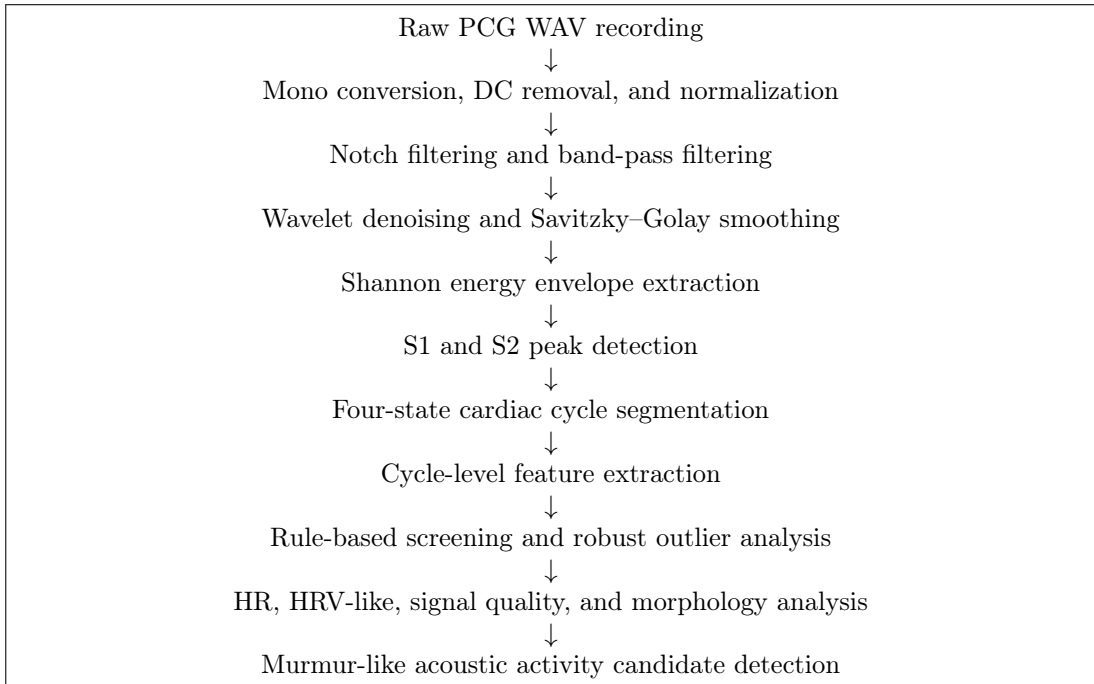


Figure 17: Summary of the PCG signal-processing pipeline.

6. Data Transmission Protocol

6.1. Experimental Comparison of Communication Methods

To select the most suitable transport for the prototype, four communication methods were implemented and compared experimentally: USB serial, Bluetooth Low Energy (BLE), Wi-Fi using UDP, and Wi-Fi using TCP. Serial was used as a wired baseline for debugging and calibration, while BLE, UDP, and TCP represent the candidate wireless transports. Although the final firmware described in Section 4 streams data over BLE, the three wireless options were evaluated using dedicated benchmark firmware variants so that their throughput, packet loss, and power behaviour could be measured under identical conditions.

To keep the comparison fair, all benchmark tests used the same configuration: a fixed batch size of 6 samples, an oversampling factor of 8, an identical structured packet format, the same receiver-side loss check based on a sequence number, and the same 2-minute acquisition window. Using the same packet format for all protocols ensures that no method appears faster simply because it transmits a smaller or simpler message.

6.1.1 Benchmark Packet Format

For the benchmark, each transmitted packet was not limited to raw samples. Instead, a small structured packet was used, as summarized in Table 6. The packet contains a header, a sequence number, a sample count, and the sample payload, giving a total length of 20 bytes.

Table 6: Structured packet format used in the communication benchmark.

Field	Size	Description
Header	2 bytes	Frame-start identifier marking the beginning of a valid packet.
Sequence number	4 bytes	Incremented per packet; used to detect receiver-side packet loss.
Sample count	2 bytes	Number of samples in the packet (fixed at 6 in the tests).
Sample payload	12 bytes	Six 16-bit ADC samples (6×2 bytes).
Total	20 bytes	Header + sequence number + count + payload.

The sequence number is the key element of this format, because it allows the receiver to detect missing packets: if the received sequence jumps from n to $n + 2$, packet $n + 1$ is counted as lost. The header marks the start of a valid packet, which is particularly useful for the continuous serial stream, while the count field makes each packet self-describing. It should be noted that this structured benchmark packet is intentionally different from the minimal streaming packet used by the final BLE firmware (Table 3), which transmits only raw ADC samples without a header or sequence number. The structured format was introduced specifically to measure packet loss during the comparison and is not part of the deployed streaming protocol.

6.1.2 Power Consumption

The power values in Table 7 were measured on the bench shown in Figure 18. The Arduino Nano ESP32 was powered from the SIGLENT SPD3303X-E supply set to a regulated 3.3 V, with the board mounted on a breadboard so that the supply rail was accessible for measurement. Because the current drawn by the ESP32 changes over time with the radio state (idle, advertising, connected, and transmitting), the oscilloscope was used to observe this dynamic behaviour, while the multimeter provided a stable DC reading. For each communication mode, the supplied voltage and the corresponding current were recorded, and the power was computed as $P = V \times I$.

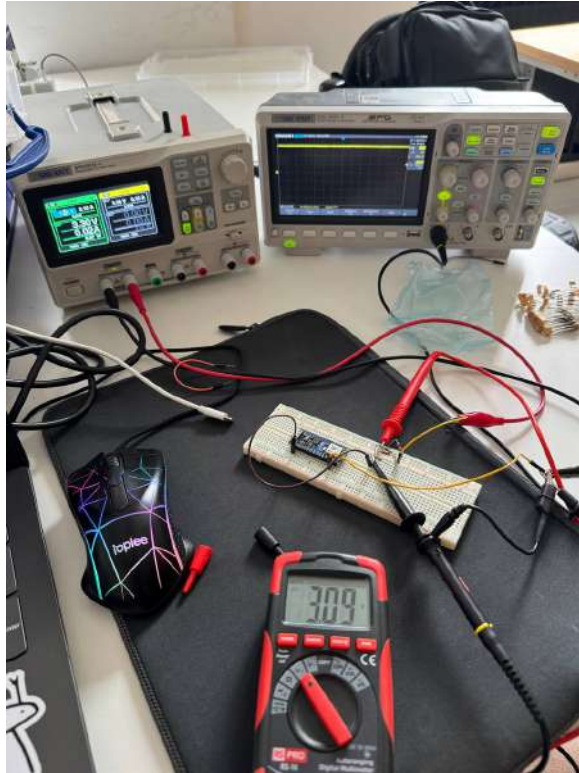


Figure 18: Laboratory bench used for the power-consumption measurements: a SIGLENT SPD3303X-E programmable DC power supply providing and displaying the regulated 3.3 V rail (left), a SIGLENT SDS1202X-E oscilloscope (right), the Arduino Nano ESP32 mounted on a breadboard, and an RS PRO RS-14 digital multimeter.

Table 7: Measured ESP32 power consumption in different communication modes.

Communication mode	Power (mW)
Serial (USB)	300
BLE connected	306
Wi-Fi TCP/UDP idle	330
BLE advertising	360
Wi-Fi TCP connected	595
Wi-Fi UDP active	600

Serial and BLE in the connected state were the lowest-power modes, while active Wi-Fi (both UDP and TCP) showed the highest consumption. The idle Wi-Fi value is lower than the active values because no client is streaming in that state. The baseline CPU and ADC cost is common to all modes, so the differences mainly reflect the additional cost of each radio.

Among the wireless options, BLE offers the lowest power consumption, which is consistent with its use in the final prototype, where low power is more important than the longer range offered by Wi-Fi.

6.1.3 BLE Advertising Interval and Power Trade-off

The benchmark above measured BLE in a single advertising configuration. Because the advertising interval directly controls how often the radio wakes up to broadcast, it is the main parameter that trades discovery speed against power in the advertising state. In the deployed firmware (Section 4) the advertising interval is set explicitly to 100 ms through `setMinInterval(0xA0)` and `setMaxInterval(0xA0)`, where the register unit is 0.625 ms ($0xA0 = 160$, $160 \times 0.625 \text{ ms} = 100 \text{ ms}$). If left unset, the library would instead use its default window of `adv_int_min = 0x20` to `adv_int_max = 0x40` (20–40 ms, effective $\approx 30 \text{ ms}$);

this default was the configuration present during the bench measurement, so its 360 mW value is the single directly measured point.

The remaining rows in Table 8, including the deployed 100 ms point, are *estimated* by extrapolating from that measured point using the standard advertising power model

$$P(T) \approx P_{\text{floor}} + \frac{E_{\text{adv}}}{T}, \quad (33)$$

where T is the advertising interval, $P_{\text{floor}} \approx 300$ mW is the radio-off baseline (equal to the measured Serial value, dominated by the CPU, ADC, and OLED), and $E_{\text{adv}} \approx 1.8$ mW-s is the per-event contribution, calibrated so that the default effective interval of ≈ 30 ms reproduces the measured 360 mW. The discovery times are typical values for a continuously scanning central and scale with the interval.

Table 8: BLE advertising power as a function of the advertising interval. Only the 20–40 ms library-default configuration was measured on the bench (360 mW); the other rows, including the deployed 100 ms setting, are extrapolated using Equation (33). Register values are in units of 0.625 ms.

Adv. interval (ms)	adv_int (min/max)	Est. power (mW)	Typical discovery	Trade-off
20 (fixed)	0x20 / 0x20	390	$\sim 20\text{--}40$ ms	Spec minimum; fastest discovery, highest power
20–40 (default)	0x20 / 0x40	360	$\sim 30\text{--}60$ ms	Library default; benchmark measurement point
100	0xA0 / 0xA0	318	$\sim 0.1\text{--}0.2$ s	Selected; set in firmware (balanced)
200	0x140 / 0x140	309	$\sim 0.2\text{--}0.4$ s	Low power, noticeable discovery delay
500	0x320 / 0x320	304	$\sim 0.5\text{--}1$ s	Marginal extra saving
1000	0x640 / 0x640	302	$\sim 1\text{--}2$ s	Slowest; near the radio-off floor

The trend follows Equation (33): shortening the interval raises the advertising-event rate and therefore the power, while lengthening it saves power but slows discovery and asymptotes toward the ≈ 300 mW radio-off floor. The absolute spread is small — about 90 mW between the fastest (20 ms) and slowest (1000 ms) settings — because the fixed CPU, ADC, and OLED load dominates the total in every case. The 100 ms interval was selected as a balanced compromise. Advertising is a brief, infrequent transient state: the device advertises only at power-up and after a disconnect, and stops as soon as the host connects, spending essentially the entire measurement session in the connected/streaming state (≈ 306 mW, Table 7). The advertising interval therefore has a negligible effect on overall battery life, so the choice carries little energy cost either way. Given this, 100 ms is preferred over the more aggressive 20–40 ms default because it lowers the advertising-state power by about 12% ($360 \rightarrow 318$ mW) while keeping discovery latency around 0.1–0.2 s, which is effectively instantaneous from the operator’s point of view when the application is opened or after an accidental disconnect. It also remains well within the BLE-recommended fast-advertising band and is reliably supported across central devices. Going to longer intervals (≥ 200 ms) would save only a few additional milliwatts while making discovery visibly slower, so 100 ms sits at the point where the responsiveness is still essentially free and the unnecessary advertising power has been trimmed.

6.1.4 Throughput and Packet-Loss Results

Each protocol was tested at increasing target sampling frequencies in order to identify its practical throughput limit. For every test, three quantities were recorded: the receiver-side packet loss (detected from skipped sequence numbers), the sender-side dropped samples (data lost inside the ESP32 when the board could not keep up with the sampling rate), and the effective delivered sample rate. The ideal packet rate for a batch size of 6 is $f_s/6$, for example 83.3 packets/s at 500 Hz and 333.3 packets/s at 2000 Hz. The results are summarized in Table 9.

Table 9: Communication benchmark results. All tests used a batch size of 6, an oversampling factor of 8, an identical 20-byte structured packet, and a 2-minute window. Here “pps” denotes packets per second, “sps” denotes delivered samples per second, “Rx” the receiver side, and “Tx” the sender (ESP32) side. The ideal packet rate is $f_s/6$.

Method	f_s (Hz)	Actual pps	Eff. (sps)	Rx loss	Tx drop	Status
Serial	500	83.3	500	0	0	OK
Serial	1000	166.7	1000	0	0	OK
Serial	2000	333.3	2000	0	28	OK
Serial	4000	344.7	2068	0	—	Overload
BLE	500	83.4	500	0	0	OK
BLE	1000	166.4	1001	0	0	OK
BLE	2000	250.7	1508	0	23207	Overload
Wi-Fi UDP	500	12.2	81	8839	0	High loss
Wi-Fi UDP	1000	12.7	82	20264	0	High loss
Wi-Fi TCP	500	83.3	500	0	177	OK
Wi-Fi TCP	1000	160.0	980	0	6096	Degraded
Wi-Fi TCP	2000	210.0	1270	0	93776	Overload

At the 500 Hz operating point used for signal acquisition and processing, Serial, BLE, and TCP all delivered the full target rate of 500 samples/s with no receiver-side loss. At higher frequencies, the practical limits of each method became visible. Serial matched the target up to 2000 Hz with negligible sender loss (28 dropped samples), but saturated at approximately 2068 samples/s when 4000 Hz was requested; this ceiling is consistent with the throughput limit of the serial link at the configured baud rate. BLE handled 500 Hz and 1000 Hz correctly, but at 2000 Hz the receiver still reported zero missing packets while the delivered rate dropped to about 1508 samples/s, indicating sender-side skipping rather than transmission loss. TCP delivered 500 Hz correctly with a small sender drop ($\approx 1.6\%$), but already showed substantial sender-side pressure at 1000 Hz ($\approx 30\%$ dropped) and overloaded at 2000 Hz.

The Wi-Fi UDP results require careful interpretation. UDP showed very high receiver-side loss even at 500 Hz, where only about 12 packets/s reached the receiver. This behaviour is most likely caused by the broadcast configuration used in the test rather than by an inherent limitation of Wi-Fi or UDP. The UDP figures should therefore be treated as a setup-related artifact, and the UDP path should be re-evaluated using unicast transmission before any conclusion is drawn about its real capability.

6.1.5 Selected Communication Method

The comparison shows that the choice of transport involves a trade-off between reliability, latency, throughput, and power. Serial is a stable wired baseline but not a wireless solution; TCP guarantees ordered delivery but at higher power and earlier sender-side saturation; UDP can offer low latency but did not guarantee delivery and, in the current setup, suffered severe loss. BLE provided reliable delivery at the 500 Hz operating point, the lowest power among the wireless options, and sufficient throughput. BLE was therefore selected as the communication method of the prototype. The BLE transport itself the GATT service, the streaming-packet format, and the acquisition data flow is described in detail in

the firmware section (Section 4).

6.2. Communication Technology

Based on the comparison presented above, BLE was adopted as the communication technology of the prototype. BLE provides low-power, short-range wireless transfer that matches the wearable use case and is sufficient for the 500 Hz acquisition rate. Wi-Fi (UDP and TCP) was not used in the final device, mainly because of its higher power consumption and, in the case of the tested UDP setup, the unreliable delivery reported above. Serial remains available as a wired link for debugging and calibration.

6.3. Packet Structure

In the deployed BLE transport, each notification carries only the raw ADC samples, without an application-level header, timestamp, sequence number, or checksum: the BLE notification boundary itself acts as the frame boundary, and delivery is handled by the BLE link layer. The detailed notification format is given in Table 3 and Figure 8 in the firmware section. The structured 20-byte packet used in the benchmark of Section 6.1 (Table 6) was a separate, measurement-only format and is not used by the deployed device.

6.4. Transmission Workflow

In the deployed BLE transport, the acquisition session is controlled through a single GATT characteristic. The host writes a START command (Table 2) to configure the sampling parameters, after which the firmware streams batches of raw ADC samples as BLE notifications until the acquisition window expires. There is no application-level acknowledgement, retransmission, or checksum; ordering and delivery are handled by the BLE link layer. The complete acquisition-to-notification data flow is described step by step in Section 4.

7. Implemented Web-Based Software Platform

In the implemented prototype, the software layer is organized as a web-based client-server application. The backend is responsible for device communication, data persistence, recording management, audio storage, and PCG analysis. The frontend provides the operator-facing interface for patient lookup, auscultation-site selection, recording control, waveform visualization, and inspection of analysis results. This separation keeps the browser interface lightweight: the frontend requests data and renders the results, while acquisition control, storage, and signal-processing operations remain on the server side.

7.1. Backend Implementation

The backend was implemented as a REST API using Python and FastAPI. The database layer is based on SQLAlchemy, while Alembic is used to manage database migrations. PostgreSQL is used to store structured information such as patients, measurement sessions, heart-sound recordings, and recording metadata. In addition to database storage, the backend saves acquired audio signals as WAV files and exposes them through API routes so that previous recordings can be played, downloaded, or analyzed later.

The backend code is organized into route, schema, model, service, and configuration layers. The route layer exposes the HTTP API, the schema layer defines request and response data structures, the model layer maps persistent database entities, and the service layer contains the main application logic. This organization improves modularity because patient management, BLE ingestion, audio storage, measurement control, and PCG analysis can be developed and tested as separate responsibilities.

The main backend responsibilities are summarized in Table 10.

The most relevant API endpoints used by the frontend are listed in Table 11.

During a recording session, the backend connects the software platform to the embedded acquisition

Table 10: Main backend responsibilities in the software platform.

Backend function	Role in the system
Patient management	Creates new patient profiles and searches existing patients by name.
Measurement state	Provides the current measurement status, selected patient context, signal quality, and live waveform samples.
BLE ingestion	Receives raw sample batches from the Arduino Nano ESP32 through Bluetooth Low Energy notifications.
Recording action	Starts a fixed-duration acquisition session, waits for the capture window to finish, and stores the resulting recording.
Audio storage	Saves the captured signal as a mono WAV file and exposes the file through an audio endpoint.
PCG analysis	Runs the saved WAV recording through the PCG processing pipeline and returns analysis results suitable for plots and tables.
Diagnostics	Provides BLE status, device scan, start, and stop endpoints to support integration and debugging.

Table 11: Main REST API endpoints used by the frontend.

Method	Endpoint	Purpose
GET	/api/patients/search	Search for existing patients by name.
POST	/api/patients	Create a new patient profile.
GET	/api/heart-measurements/current	Retrieve the current measurement state for a patient.
POST	/api/heart-measurements/{patientId}/record	Start and store a fixed-duration heart-sound recording.
GET	/api/heart-recordings/{recordingId}/audio	Retrieve the saved WAV audio file.
GET	/api/heart-recordings/{recordingId}/analysis	Run or retrieve PCG analysis results for a saved recording.
DELETE	/api/heart-recordings/{recordingId}	Delete a stored recording and its generated audio files.
GET/POST	/api/ble/...	Inspect, start, stop, and debug BLE communication.

device. The Arduino Nano ESP32 streams raw samples using BLE notifications. The backend collects these samples during a fixed acquisition window, converts the received data into a WAV recording, stores the metadata in the database, and makes the recording available through the REST API. If BLE data are unavailable during development, the backend can still expose a mock or idle waveform, which is useful for testing the frontend without the physical sensor.

7.2. Frontend Implementation

The frontend was implemented as a React application built with Vite. The interface is component-based and is organized around two main pages: a patient intake page and a measurement page. The patient intake page allows the operator to search for an existing patient, create a new patient profile, select the active patient, and inspect the previous recording history. After a patient is selected, the measurement page provides the live measurement workspace.

The measurement page includes a cardiac auscultation map with the standard listening sites: aortic, pulmonic, Erb’s point, tricuspid, and mitral/apex. The user selects the auscultation area before starting

a recording. During recording, the selected site is locked to avoid changing the anatomical context while samples are being acquired. The page also shows the current session state, signal quality, elapsed recording time, live waveform information, and the list of stored recordings for the selected patient.

The frontend communicates with the backend using HTTP requests. The API base URL is configurable through the environment variable `VITE_API_BASE_URL`, while the default development backend address is `http://localhost:8000`. The frontend periodically polls the current measurement endpoint to keep the displayed waveform and session status synchronized with the server. When the user presses the record button, the frontend sends a recording request to the backend and waits until the fixed-duration capture finishes. After the recording is stored, the frontend refreshes the patient history and displays the new recording.

The analytics tab is used for post-acquisition inspection. Instead of computing the PCG analysis directly in the browser, the frontend sends a request to the backend analysis endpoint for a selected recording. The backend returns signal-processing results and plot-ready data, while the frontend renders the waveform, envelope, detected peaks, segmentation information, cycle-level measurements, and screening indicators. This design keeps the computationally heavier PCG processing on the backend and allows the frontend to focus on interaction and visualization.

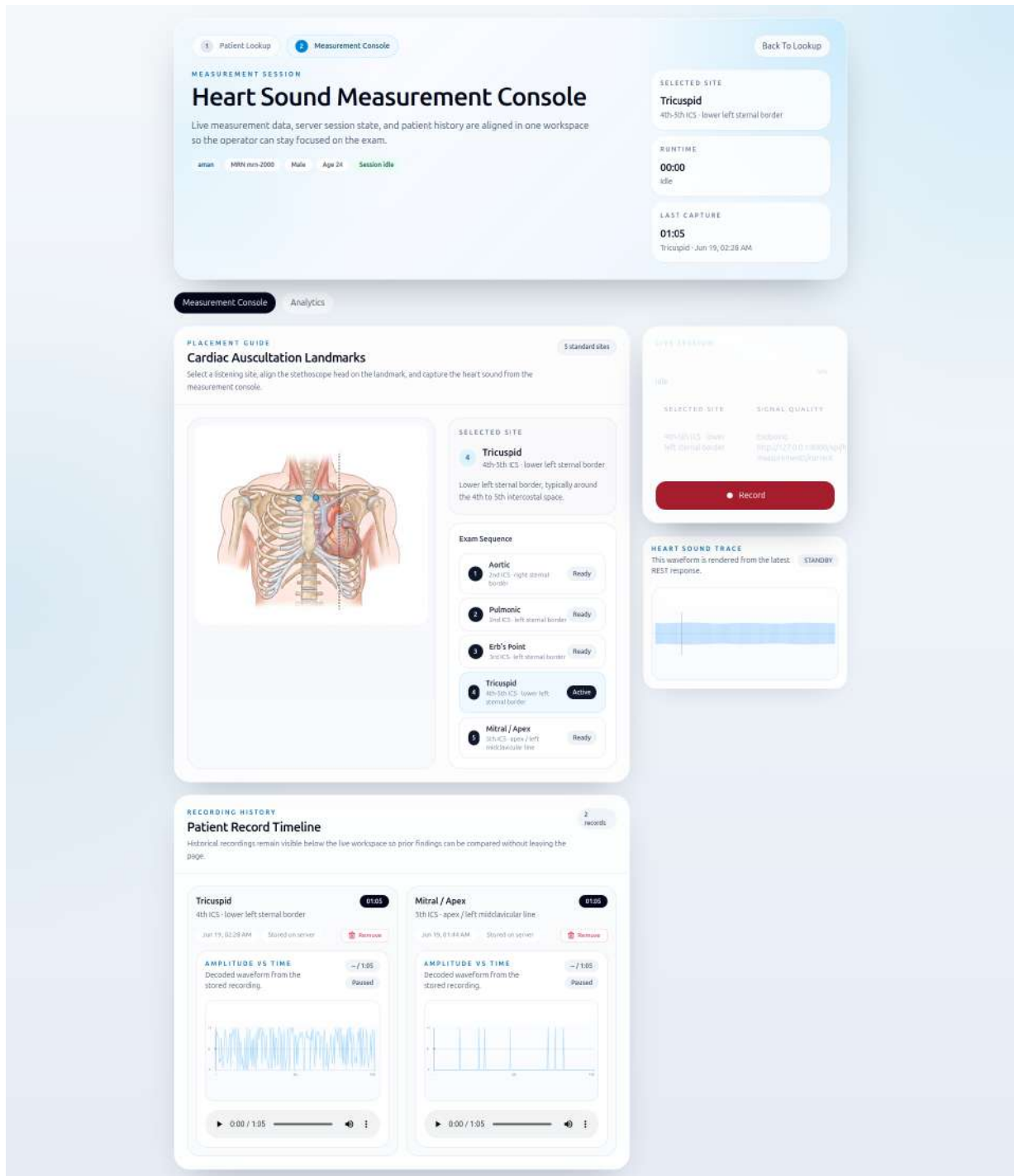


Figure 19: Frontend measurement console implemented in React. The interface shows the selected patient, active auscultation site, live recording controls, waveform preview, session status, and stored recording history.

7.3. Software Data Flow

The complete software workflow is summarized below:

1. The operator searches for an existing patient or creates a new patient profile in the frontend.
2. The frontend requests patient and recording information from the backend REST API.
3. The operator selects the auscultation site using the graphical heart landmark map.
4. The frontend sends a recording request to the backend for the selected patient and site.
5. The backend starts the BLE acquisition process and collects raw samples from the Arduino Nano ESP32.

6. At the end of the acquisition window, the backend stores the recording metadata in PostgreSQL and saves the signal as a WAV file.
7. The frontend refreshes the measurement state and displays the new recording in the patient history.
8. When analysis is requested, the backend runs the PCG processing pipeline on the saved WAV file.
9. The frontend renders the returned waveform, segmentation, feature values, and screening outputs.

This workflow connects the embedded acquisition layer with the user-facing medical interface. The backend acts as the bridge between the BLE sensor and the persistent software platform, while the frontend provides an accessible interface for operating the prototype, reviewing previous recordings, and visualizing the PCG analysis results.

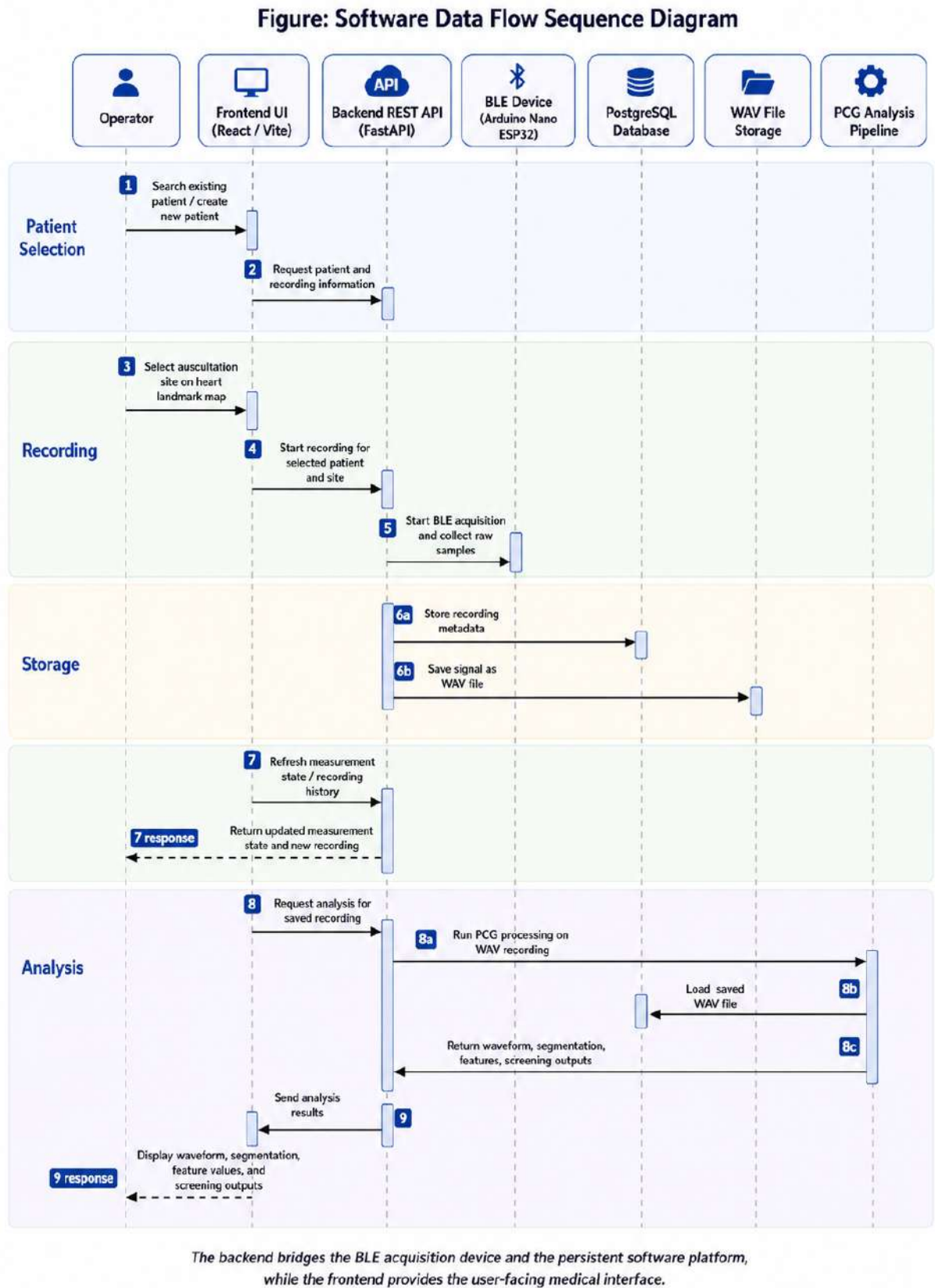


Figure 20: Software data flow sequence diagram of the implemented IoMT phonocardiography platform.

8. Results

8.1. Signal-Processing Test Results

8.1.1 Recording and Detection Summary

The proposed signal-processing pipeline was applied to a PCG recording with a duration of 24.92 s and a sampling rate of 500 Hz. After preprocessing, filtering, and Shannon envelope extraction, the S1 and S2 peaks were detected using physiological timing constraints.

For this recording, 29 S1 peaks and 29 S2 peaks were detected. These peaks formed 29 accepted S1–S2 pairs and 29 complete cardiac cycles. No candidate cycles were rejected during the cycle extraction stage, resulting in a cycle acceptance rate of 100%. The main detection and cycle extraction results are summarized in Table 12.

Table 12: Summary of PCG recording, S1/S2 detection, and cardiac cycle extraction.

Parameter	Value
Recording duration	24.92 s
Sampling rate	500 Hz
Detected S1 peaks	29
Detected S2 peaks	29
Accepted S1–S2 pairs	29
Accepted cardiac cycles	29
Rejected candidate cycles	0
Cycle acceptance rate	100%
Estimated heart rate from S1–S1 intervals	73.4 bpm

The high cycle acceptance rate indicates that the S1/S2 detection and physiological pairing steps produced a consistent sequence of cardiac cycles for this recording. The estimated heart rate from the S1–S1 intervals was 73.4 bpm, which is consistent with the per-cycle heart rate statistics reported in the following sections.

8.1.2 Per-Cycle Feature Results

After cardiac cycle segmentation, temporal and energy-based features were extracted from each accepted cardiac cycle. In total, 29 accepted cycles were analysed. The extracted features included heart rate, S1 duration, S2 duration, systolic interval, diastolic interval, S1/S2 amplitude ratio, and systole-to-diastole duration ratio.

The per-cycle feature statistics are summarized in Table 13. The median heart rate was 73.17 bpm, while the mean heart rate was 71.31 bpm. The S1 and S2 durations were relatively stable, with median values of 102.00 ms and 84.00 ms, respectively. The systolic and diastolic intervals showed more variability, especially the diastolic interval, which is expected because changes in cycle duration are often reflected more strongly in the diastolic phase.

Table 13: Per-cycle feature statistics extracted from the accepted cardiac cycles.

Feature	Mean	Median	Std.	Range
Heart rate (bpm)	71.31	73.17	7.90	45.80–79.16
S1 duration (ms)	99.10	102.00	14.99	42.00–122.00
S2 duration (ms)	84.69	84.00	2.69	80.00–90.00
Systolic interval (ms)	313.24	306.00	43.29	236.00–466.00
Diastolic interval (ms)	542.34	516.00	92.22	462.00–854.00
S1/S2 amplitude ratio	0.69	0.61	0.27	0.43–1.78
Systole/diastole ratio	0.58	0.59	0.05	0.41–0.68

Per-Cycle Feature Distributions for Accepted Cycles

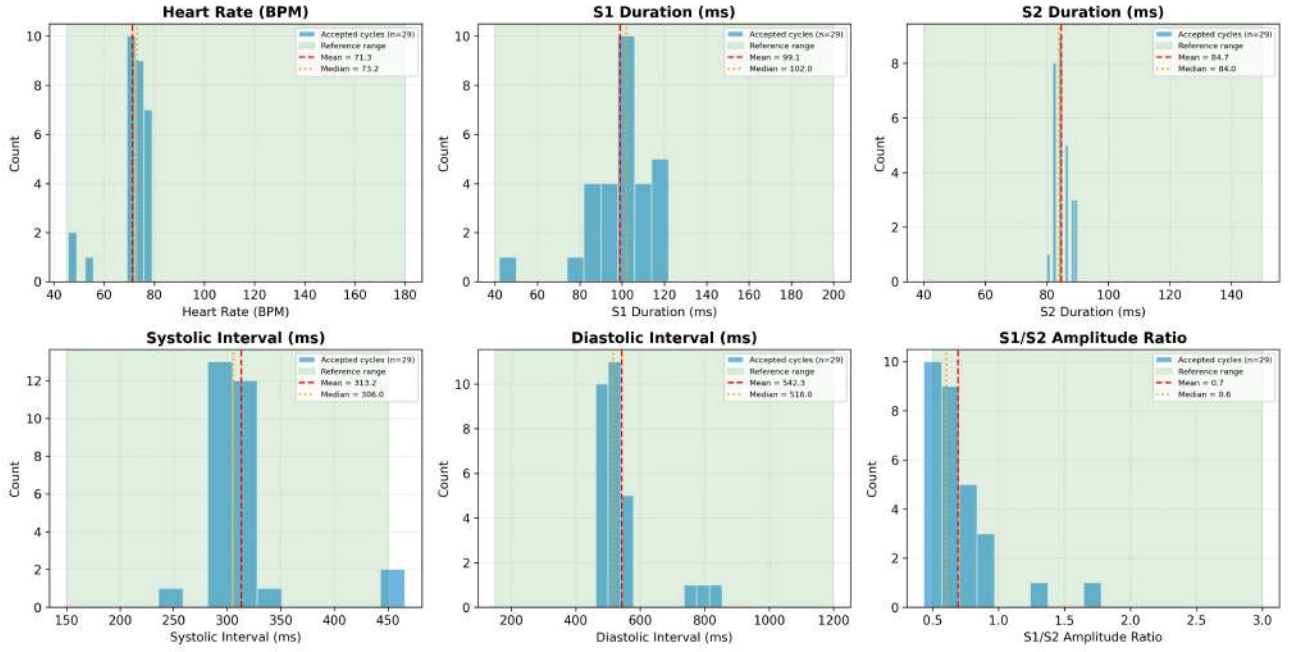


Figure 21: Distribution of the main per-cycle features extracted from the accepted cardiac cycles. The dashed lines indicate the mean and median values, while the shaded regions show the reference ranges used for rule-based screening.

Figure 21 shows the distribution of the extracted features across the accepted cycles. Most feature values are concentrated within the expected reference ranges. A small number of cycles show lower heart-rate values or lower S1/S2 amplitude ratios, which are further examined in the rule-based screening step.

8.1.3 Rule-Based Screening Results

After feature extraction, the accepted cardiac cycles were evaluated using the predefined rule-based screening ranges. This step was used to identify cycles with feature values outside the expected ranges. These cycles were not considered diagnostic abnormalities; they were only flagged as signal-derived cycles requiring further inspection.

Out of the 29 accepted cardiac cycles, 22 cycles were within the expected feature ranges, while 7 cycles were flagged by the rule-based screening step. This corresponds to 75.9 percent normal cycles and 24.1 percent flagged cycles. The flagged cycles were mainly related to either a slightly prolonged systolic interval or a low S1/S2 amplitude ratio. The summary of the screening results is shown in Table 14.

Table 14: Summary of rule-based cycle screening results.

Screening output	Value
Accepted cardiac cycles	29
Normal cycles	22/29
Normal cycle percentage	75.9%
Flagged cycles	7/29
Flagged cycle percentage	24.1%

Most flagged cycles were related to the S1/S2 amplitude ratio, indicating that S1 was weaker than S2 in those cycles. Two cycles were flagged because their systolic interval was slightly above the selected threshold. These findings were treated as signal-quality or cycle-pattern indicators and were not interpreted as clinical abnormalities.

8.1.4 Heart Rate and HRV-Like Results

The detected S1 peaks were used to estimate the heart rate and short-window HRV-like metrics. In total, 29 S1 peaks were detected, producing 28 S1–S1 intervals. All 28 intervals were initially considered valid based on the broad physiological interval range.

The mean heart rate was 71.3 bpm, while the median heart rate was 73.4 bpm. The difference between the mean and median values was mainly caused by a small number of longer S1–S1 intervals, which reduced the mean heart rate. The main HRV-like results are summarized in Table 15.

It is important to note that these metrics are derived from PCG-based S1–S1 intervals, not from ECG R–R intervals. Therefore, they are reported as short-window HRV-like metrics and should not be interpreted as clinical ECG-based HRV estimates.

Table 15: Short-window HRV-like metrics derived from PCG S1–S1 intervals.

Metric	Value
Detected S1 peaks	29
Available S1–S1 intervals	28
Valid S1–S1 intervals	28
Mean NN interval	856.4 ms
Median NN interval	817.0 ms
SDNN	132.9 ms
RMSSD	144.7 ms
NN50	12
pNN50	44.4%
CVNN	15.5%
Mean heart rate	71.3 bpm
Median heart rate	73.4 bpm
Heart-rate variability SD	8.1 bpm

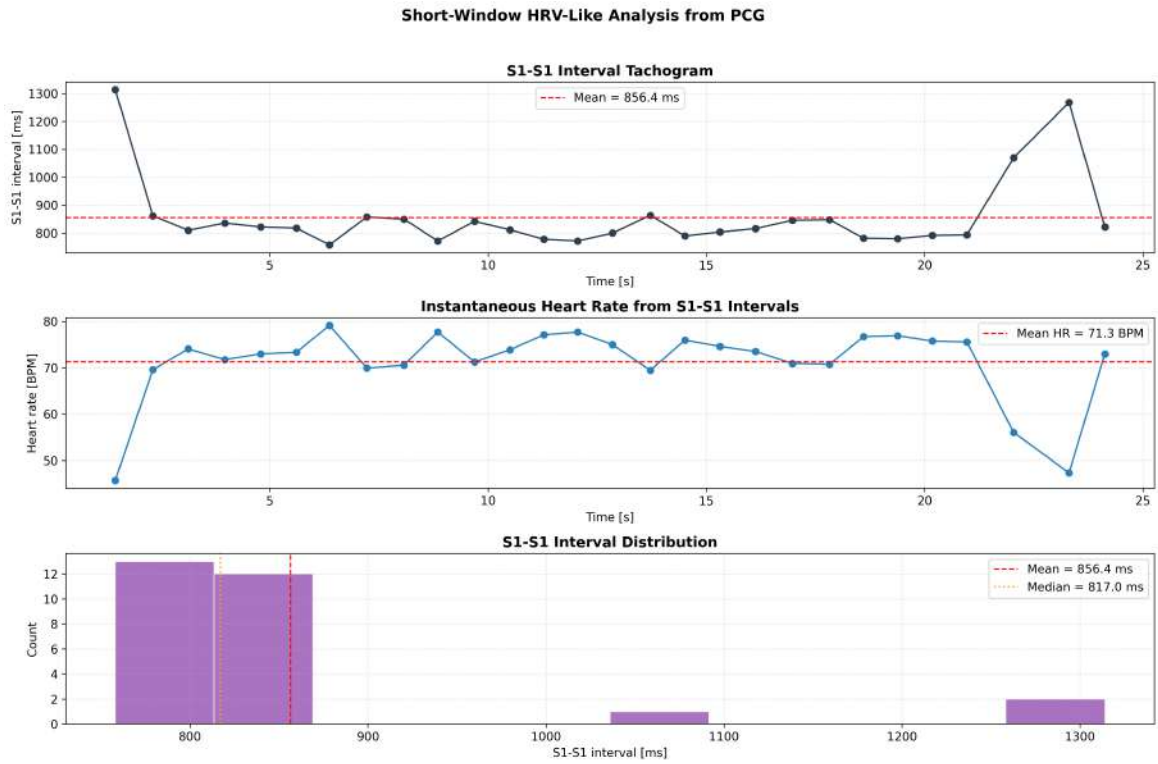


Figure 22: Short-window HRV-like analysis from PCG-derived S1–S1 intervals. The plots show the S1–S1 interval tachogram, instantaneous heart rate, and S1–S1 interval distribution.

Figure 22 shows that most S1–S1 intervals are concentrated around 800–850 ms, corresponding to a heart rate of approximately 70–75 bpm. A small number of longer intervals are visible and increase the variability metrics.

To reduce the effect of suspicious or uncertain S1–S1 intervals, a robust artifact-rejection step was applied using a median-based accepted interval range. The accepted range was 612.8–1021.3 ms. After this filtering step, 25 out of 28 intervals were retained and 3 intervals were rejected.

The robust HRV-like results are summarized in Table 16. After artifact rejection, the mean NN interval became 813.1 ms and the mean heart rate became 73.9 bpm. The SDNN and RMSSD values were also reduced, indicating that the rejected intervals were responsible for a large part of the variability in the raw HRV-like results.

Table 16: Robust HRV-like metrics after artifact rejection of suspicious S1–S1 intervals.

Metric	Value
Artifact rejection range	612.8–1021.3 ms
Intervals kept	25/28
Intervals rejected	3/28
Mean NN interval	813.1 ms
Median NN interval	812.0 ms
SDNN	30.7 ms
RMSSD	44.4 ms
pNN50	33.3%
Mean heart rate	73.9 bpm
Median heart rate	73.9 bpm

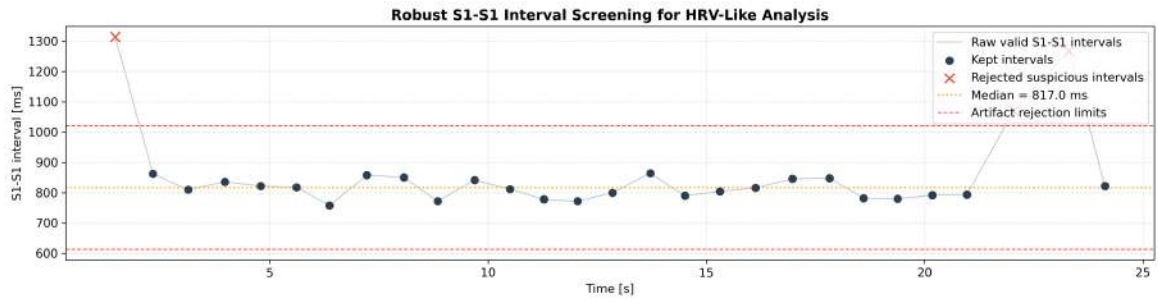


Figure 23: Robust screening of S1–S1 intervals for HRV-like analysis. Suspicious intervals outside the median-based accepted range were rejected before recalculating the robust HRV-like metrics.

Figure 23 shows the retained and rejected S1–S1 intervals. The rejected intervals correspond to unusually long cycle intervals, which may be caused by missed or uncertain S1 detections. Therefore, the robust HRV-like values provide a more stable summary of the recording.

8.1.5 Murmur-Like Acoustic Activity Results

The systolic and diastolic regions of each accepted cardiac cycle were analysed to detect signal-derived murmur-like acoustic activity candidates. The activity ratios were compared with the selected detection threshold of 0.35.

For the analysed recording, no systolic murmur-like activity was detected. Therefore, the number of systolic active cycles was 0 out of 29. No early systolic-like, mid systolic-like, late systolic-like, holosystolic-like, diamond-like, or plateau-like systolic patterns were observed.

In the diastolic analysis, 2 out of 29 cycles showed mid-diastolic acoustic activity candidates, corresponding to 6.9 percent of the accepted cycles. No late-diastolic activity was detected. The summary of the murmur-like acoustic activity results is shown in Table 17.

Table 17: Summary of signal-derived murmur-like acoustic activity results.

Activity type	Result
Analysed cardiac cycles	29
Systolic active cycles	0/29
Systolic active percentage	0.0%
Early systolic-like cycles	0
Mid systolic-like cycles	0
Late systolic-like cycles	0
Holosystolic-like cycles	0
Diamond-like systolic cycles	0
Plateau-like systolic cycles	0
Mid-diastolic-like cycles	2
Late-diastolic-like cycles	0
Mid/Late diastolic active cycles	2/29
Mid/Late diastolic active percentage	6.9%

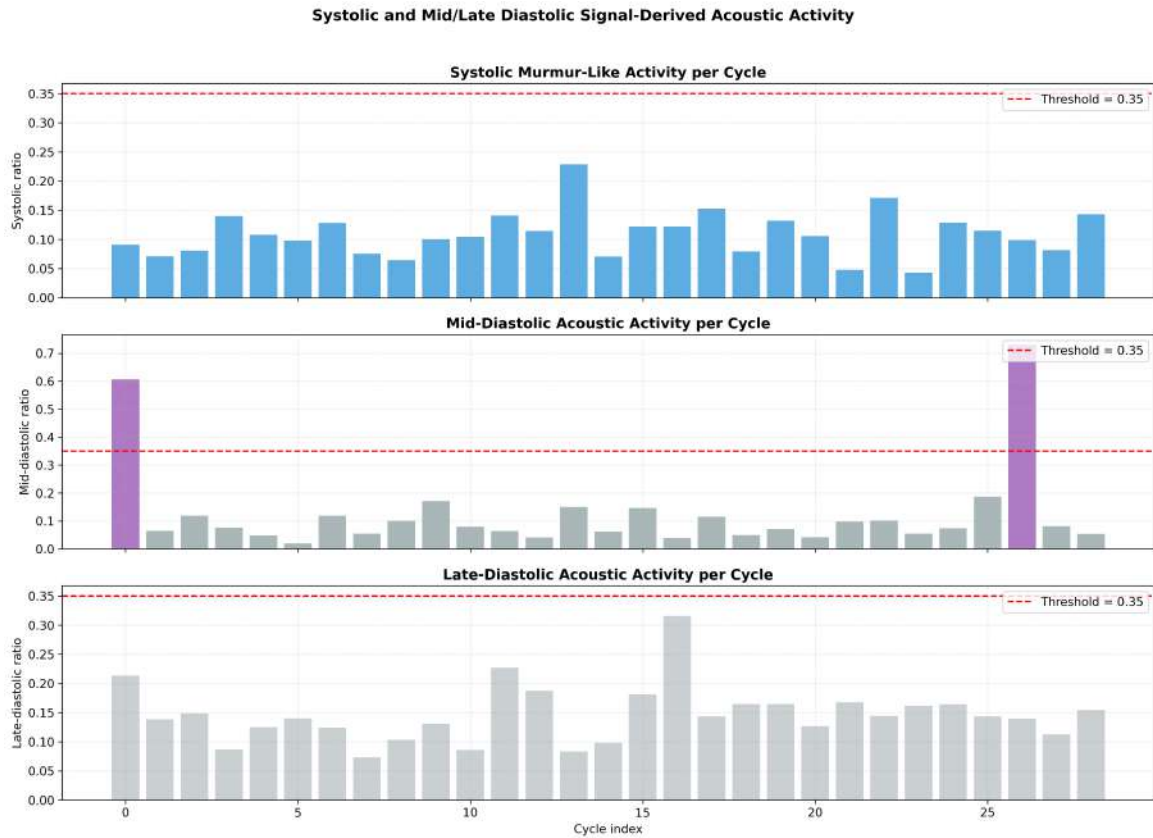


Figure 24: Signal-derived systolic and mid/late diastolic acoustic activity ratios for each accepted cardiac cycle. The dashed red line represents the selected detection threshold.

Figure 24 shows that the systolic activity ratios remained below the detection threshold for all accepted cycles. In contrast, two cycles crossed the threshold in the mid-diastolic region. These results indicate weak mid-diastolic acoustic activity candidates in a small portion of the recording. However, these findings are signal-derived candidates only and should not be interpreted as clinical diagnoses.

8.1.6 Signal Quality and Morphology Results

The quality of the filtered PCG signal was evaluated using frequency-domain descriptors and morphology consistency metrics. The filtering stage strongly reduced the out-of-band energy, indicating that the preprocessing pipeline successfully suppressed unwanted frequency components while preserving the main PCG frequency range. The raw out-of-band energy ratio was 95.76%, while the filtered out-of-band energy ratio was reduced to 3.23%. This corresponds to an out-of-band reduction of 96.63%. The dominant frequency of the filtered PCG signal was 35.43 Hz, and the spectral centroid was 39.46 Hz. These values indicate that most of the useful acoustic energy was concentrated in the lower PCG frequency range.

The signal quality results are summarized in Table 18.

Table 18: Frequency-domain signal quality metrics of the filtered PCG signal.

Metric	Value
Raw out-of-band energy ratio	95.76%
Filtered out-of-band energy ratio	3.23%
Out-of-band energy reduction	96.63%
Dominant frequency	35.43 Hz
Spectral centroid	39.46 Hz
Spectral bandwidth	9.54 Hz
Spectral entropy	0.733
Low-band energy, 25–60 Hz	97.30%
Mid-band energy, 60–120 Hz	2.61%
High-band energy, 120–200 Hz	0.10%

The morphology consistency analysis was performed by aligning all accepted cardiac cycles around S1 and comparing each cycle with the mean cycle template. All 29 accepted cycles were used in this analysis. The mean template similarity was 0.240, while the median template similarity was 0.351. Using a low-similarity threshold of 0.75, 26 out of 29 cycles were marked as low-similarity cycles.

These results indicate that although the S1 and S2 timing structure was detected consistently, the detailed waveform morphology varied across cycles. This may be due to noise, sensor movement, amplitude variation, segmentation uncertainty, or natural cycle-to-cycle variability. Therefore, morphology similarity is interpreted only as a signal-quality descriptor, not as a clinical indicator.

The morphology consistency results are summarized in Table 19.

Table 19: Cycle morphology consistency results based on S1-aligned cardiac cycles.

Metric	Value
Aligned cycles used	29/29
Template window	-0.10 to 0.82 s around S1
Mean template similarity	0.240
Median template similarity	0.351
Template similarity standard deviation	0.425
Low-similarity threshold	0.75
Low-similarity cycles	26

The spectrogram and morphology plots presented earlier provide visual support for these numerical results by showing the time-frequency structure of the filtered PCG signal and the cycle-to-template similarity pattern.

8.2. Discussion

From the signal-processing perspective, the results show that the acquired PCG recording could be filtered, segmented, and analysed to extract meaningful signal-derived descriptors. The pipeline detected a complete sequence of cardiac cycles and provided heart-rate, HRV-like, signal-quality, morphology, and murmur-like acoustic activity information. However, these outputs should be interpreted only as signal-processing descriptors and not as clinical diagnoses. Further validation with longer recordings, repeated acquisitions, and expert-labelled datasets would be required to assess the reliability of the method in a medical context.

9. Conclusions

This report presented the design and implementation plan of a digital phonocardiography system for cardiac sound acquisition, processing, transmission, and visualization. The system integrates hardware design, firmware development, signal processing, wireless communication, and software implementation into a complete Internet of Medical Things prototype.

The main achievements of the project are:

- design of a dedicated acquisition hardware architecture;
- implementation of firmware for sensor data acquisition;
- development of a signal-processing pipeline for PCG signals;
- definition of a data transmission protocol;
- implementation of a software interface for visualization and analysis.

Future improvements may include miniaturization of the hardware, battery optimization, improved filtering and segmentation algorithms, real-time classification of abnormal heart sounds, and validation on a larger dataset.

References

A. Signal Processing Parameters

Table 20: Main signal-processing parameters used in the PCG analysis pipeline.

Parameter	Value
Sampling rate	500 Hz
Band-pass range	25–200 Hz
Notch frequencies	50, 100, 150 Hz
Notch Q factor	35
Filter order	4
Wavelet	db6
Wavelet level	4
Wavelet threshold scale	0.6
Envelope cutoff	8 Hz
Minimum peak distance	0.25 s
S1–S2 valid range	0.07–0.50 s
S2–S1 valid range	0.18–1.40 s
Murmur-like detection threshold	0.35

Acknowledgements

The authors would like to thank the lecturers and teaching staff of the Laboratory of Internet of Medical Things course for their guidance and support during the development of this project.